



EFREI Paris

Master 2 Data Engineering & IA

RNCP36739 — Expert en Ingénierie de Données

Rapport écrit de certification

Épreuve certifiante – Évaluation des compétences

Architecture de données : stockage, traitement massif, cloud et IA

Auteurs : Julien TREMONT-RAIMI

Antoine VANDEPLANQUE

Promotion : 2024–2026

Formation : Master 2 Data Engineering & IA

Établissement : EFREI Paris

16 juin 2026

Table des matières

Remerciements	1
1 Introduction	1
1.1 Contexte de la certification	1
1.2 Périmètre et projets présentés	1
1.3 Méthodologie de rédaction	1
1.4 Conventions typographiques	2
1.5 Stack technique transversale	2
1.6 Organisation du rapport	2
2 Présentation d'entreprise	3
2.1 Entreprise	3
2.1.1 Geismar	3
2.2 Le poste : Alternant Data Engineer	3
2.2.1 Contexte	3
2.2.2 Missions	3
2.2.3 Environnement technique	3
3 Synthèse des blocs de compétences RNCP36739	3
4 Bloc 1 — Concevoir et développer une architecture de stockage de données	4
4.1 Compétences visées	4
4.2 Contexte et enjeux	4
4.2.1 ACID vs BASE — les deux paradigmes du stockage	4
4.3 Analyse comparative des modèles de stockage	5
4.4 Panorama de nos réalisations	5
4.5 Jobtech — Architecture Medallion Bronze → Silver → Gold	6
4.5.1 Contexte	6
4.5.2 Architecture	6
4.5.3 Gestionnaire de connexions MySQL	6
4.5.4 Insertion en masse	7
4.5.5 Intégration multi-sources — gestion de la hétérogénéité	8
4.5.6 Couche Bronze — Parquet optimisé	8
4.5.7 Pipeline Silver — transformation et double stockage	9
4.5.8 Résultats chiffrés	9
4.5.9 API Gold — Django REST Framework	10
4.5.10 Modélisation du schéma relationnel — normalisation et contraintes	11
4.6 project_cloud — Architecture NoSQL sur AWS	11
4.6.1 Contexte	11
4.6.2 Architecture	12
4.6.3 Global Secondary Index — accès O(1)	12
4.6.4 Couche Gold — 10 tables d'agrégats pré-calculés	13
4.6.5 Résultats chiffrés	13
4.6.6 Politique de capacité DynamoDB et gestion du throttling	14
4.6.7 Structure de données et sérialisation DynamoDB	14
4.7 Synthèse des compétences démontrées	14
5 Bloc 2 — Concevoir, développer et déployer une solution de traitement des données massives	15
5.1 Compétences visées	15
5.2 Contexte et enjeux	15
5.3 Panorama de nos réalisations	15
5.4 projet_spark — Traitement massif distribué sur cluster Spark	16
5.4.1 Contexte	16
5.4.2 Topologie du cluster Spark	16
5.4.3 Architecture	16
5.4.4 Analyse exploratoire et volumétrie	17

5.4.5	Ingestion batch — Bronze météo Open-Meteo	17
5.4.6	Ingestion streaming — Bronze vols	18
5.4.7	Transformation Silver — normalisation et unification temporelle	18
5.4.8	Modèle d'exécution Spark Structured Streaming	19
5.4.9	Couche Gold — cinq analyses Spark SQL vers MySQL	19
5.4.10	Les cinq analyses Gold — contenu et valeur métier	20
5.4.11	Résultats chiffrés	20
5.4.12	Adaptive Query Execution — mécanismes et gains	21
5.4.13	Hive Metastore — persistance et gouvernance Silver	21
5.4.14	Optimisation du pipeline Bronze — partitionnement et pushdown	22
5.5	fizzbuzz-Efrei — Chaîne DevOps CI/CD et Containerisation	22
5.5.1	Contexte	22
5.5.2	Pipeline CI/CD	22
5.5.3	Containerisation Docker production-grade	23
5.5.4	Résultats chiffrés	24
5.5.5	Docker Compose — orchestration multi-conteneurs	24
5.5.6	Stratégie de sécurité de l'image Docker	25
5.5.7	Analyse de couverture et qualité du code	25
5.6	Synthèse des compétences démontrées	26
6	Bloc 3 — Implémenter et optimiser des solutions de stockage et de traitement de données sur le cloud	26
6.1	Compétences visées	26
6.2	Contexte et enjeux	27
6.3	Contexte et enjeux	27
6.3.1	Positionnement dans l'écosystème AWS	27
6.4	Architecture cloud	28
6.5	Stockage S3 — lac de données brutes horodaté	28
6.6	DynamoDB — modèle d'accès et GSI	29
6.6.1	Modèle de données	29
6.6.2	Global Secondary Index — requête O(1) sur le dernier snapshot	30
6.6.3	Ingestion incrémentale depuis S3	30
6.7	Agrégation — 10 tables analytiques	31
6.8	Performance Lambda — cold start et optimisation	31
6.8.1	Cold start Lambda et stratégies de mitigation	31
6.8.2	Gestion des dépendances Lambda — layers	32
6.9	Lambda et API Gateway — 13 endpoints serverless	32
6.10	Résultats et indicateurs	33
6.11	Sécurité et gestion des identités (IAM)	33
6.11.1	Principe du moindre privilège sur AWS	33
6.11.2	Chiffrement et protection des données	34
6.11.3	Automatisation de l'ingestion par EventBridge Scheduler	34
6.12	Optimisation des coûts cloud	34
6.12.1	Modèle de tarification serverless vs serveur permanent	34
6.12.2	Optimisation du stockage S3	35
6.13	Surveillance et observabilité	35
6.13.1	CloudWatch Metrics — tableau de bord opérationnel	35
6.14	Synthèse des compétences démontrées	35
7	Bloc 4 — Implémenter des méthodes d'intelligence artificielle	36
7.1	Compétences visées	36
7.2	Contexte et enjeux	36
7.3	Architecture et cycle ML	37
7.4	Analyse exploratoire du domaine	37
7.4.1	Contexte industriel de la maintenance prédictive	37
7.5	Préparation des données et pipeline de transformation	37
7.5.1	Features et cible	37
7.5.2	Pipeline sklearn	38
7.6	Entraînement comparatif et sélection du meilleur modèle	39

7.6.1	Architecture du benchmark	39
7.6.2	HistGradientBoostingClassifier — algorithme et avantages	40
7.6.3	Modèles candidats	40
7.7	Résultats et évaluation	40
7.7.1	Tâche 1 — Classification binaire : failure_within_24h	41
7.7.2	Tâche 2 — Classification multiclasse : failure_type (5 classes)	41
7.7.3	Interprétabilité — Importance des variables	41
7.8	Déploiement et reproductibilité	42
7.8.1	Gestion des artefacts ML	42
7.8.2	Configuration centralisée — TASK_CONFIGS	42
7.9	Mise en service — FastAPI et Streamlit	42
7.9.1	API de prédiction FastAPI	42
7.9.2	Tableau de bord Streamlit — EDA et communication visuelle	43
7.9.3	Tests automatisés	43
7.10	Analyse des erreurs et matrice de confusion	44
7.10.1	Tâche binaire — analyse précision vs rappel	44
7.10.2	Tâche multiclasse — analyse par classe	44
7.11	Robustesse et validation croisée	44
7.11.1	Choix du split stratifié vs k-fold cross-validation	44
7.11.2	Interprétabilité — limites de la permutation importance	45
7.12	Synthèse des compétences démontrées	45
8	Bloc 5 — Concevoir une stratégie de management et de gouvernance de données pour transformer les données en informations créatrices de valeur	45
8.1	Compétences visées	46
8.2	Contexte et enjeux	46
8.3	Gouvernance des données — rôles, responsabilités, catalogue	46
8.3.1	Modèle de rôles RACI appliqué aux projets	46
8.3.2	Catalogue de données et lignage	46
8.4	Qualité des données — audit et remédiation	47
8.4.1	Dimensions de la qualité	47
8.4.2	Score de qualité Silver — Jobtech	47
8.4.3	Plan de remédiation	47
8.5	Conformité et réglementation	48
8.5.1	RGPD et données personnelles	48
8.5.2	Traçabilité et auditabilité	48
8.6	Stratégie de sécurité des données	48
8.6.1	Matrice de sécurité par couche	48
8.6.2	Politique d'accès aux données	48
8.7	Synthèse des compétences démontrées	49
9	Conclusion	49
9.1	Fil conducteur — du stockage à la valeur métier	49
9.2	Synthèse des compétences acquises	49
9.3	Progression observée sur deux années	50
9.4	Prise de recul — limites et axes d'amélioration	50
9.5	Ouverture	50

Remerciements

Nos remerciements les plus chaleureux s'adressent tout particulièrement à nos intervenants et professeurs. Nous souhaitons souligner la richesse et la qualité exceptionnelle de leurs enseignements, qui ont grandement enrichi nos connaissances et affûté nos compétences. Leur encadrement de qualité, leur patience et leurs précieux conseils prodigués avec bienveillance tout au long de cette formation ont été des piliers essentiels de notre progression. Nous sommes reconnaissants de leur disponibilité et de leur expertise, qui ont su nous inspirer et nous guider vers l'excellence.

Nous remercions également chaleureusement l'ensemble de l'équipe pédagogique et administrative. Leur soutien logistique constant, leur organisation rigoureuse et leur disponibilité sans faille, souvent dans l'ombre, ont permis d'assurer un environnement de travail serein et efficace. Leur réactivité face à nos sollicitations a été un atout majeur dans le bon déroulement de ce parcours.

Enfin, nous adressons nos vifs remerciements à l'ensemble des personnes de Geismar. Nous sommes reconnaissants pour le cadre d'apprentissage stimulant qu'ils nous ont offert. Cet environnement propice à l'acquisition de savoir-faire pratiques et à la mise en application concrète des théories a été déterminant pour notre développement professionnel. Leur confiance et les opportunités d'expérimentation qu'ils nous ont accordées ont été d'une valeur inestimable.

À tous, merci pour votre contribution essentielle à la concrétisation de ce projet.

1 Introduction

1.1 Contexte de la certification

Ce rapport constitue le dossier écrit de certification professionnelle RNCP36739 « Expert en Ingénierie de Données », niveau 7 (Bac+5). Il est produit dans le cadre du Master 2 Data Engineering & IA d'EFREI Paris (promotion 2024–2026) et vise à démontrer l'acquisition des cinq blocs de compétences du référentiel [1]. Le dossier couvre des projets réalisés sur deux années de formation, incluant des projets académiques sur des datasets réels et des travaux pratiques ciblant les technologies industrielles du Big Data.

1.2 Périmètre et projets présentés

Quatre projets constituent le cœur de ce rapport :

Projet	Domaine	Blocs couverts
Jobtech	Agrégation données d'emploi tech européen	Bloc 1 (principal), Bloc 5
project_cloud	Plateforme temps réel données Paris (AWS)	Bloc 1 (NoSQL), Bloc 3, Bloc 5
projet_spark	Pipeline Big Data aéronautique US	Bloc 2 (principal)
fizzbuzz-Efrei	Chaîne CI/CD et containerisation	Bloc 2 (DevOps)
data_science_project	Maintenance prédictive industrielle (ML)	Bloc 4 (principal)

Cette répartition est le résultat d'une analyse de la couverture des compétences : chaque projet est assigné au bloc qu'il démontre le mieux, en évitant la redondance. Les dimensions transversales (gouvernance, qualité, sécurité) sont regroupées dans Bloc 5.

1.3 Méthodologie de rédaction

Chaque section de bloc suit une structure identique pour faciliter la lecture et la comparaison :

1. **Compétences visées** : les sous-compétences du référentiel RNCP directement adressées par le projet.

2. **Contexte et enjeux** : le problème métier ou technique que le projet résout, et pourquoi ce problème justifie les choix réalisés.
3. **Architecture** : schémas ASCII et descriptions des composants, avec justification des choix par rapport aux alternatives.
4. **Réalisations techniques** : extraits de code réels commentés, tableaux de métriques, résultats mesurés.
5. **Axes d'amélioration** : prise de recul honnête sur les limites de l'implémentation et les évolutions possibles.
6. **Synthèse et lien RNCP** : correspondance explicite entre les réalisations et les sous-compétences du référentiel.

1.4 Conventions typographiques

Les extraits de code sont présentés dans des blocs `lstlisting` avec indication du fichier source en légende. Les schémas ASCII illustrent les architectures sans recourir à des outils de diagramme externes. Les placeholders

[Capture] dans <fichier> lignes X à Y

indiquent les emplacements prévus pour les captures d'écran lors de la finalisation du dossier. Les références bibliographiques sont numérotées selon le standard Vancouver et listées en fin de document.

1.5 Stack technique transversale

Les cinq projets partagent un socle technologique commun qui témoigne d'une cohérence dans les choix d'outillage :

Domaine	Outil principal	Usage dans les projets
Langage	Python 3.9–3.12	Tous les projets
Big Data	Apache Spark 3.x (PySpark)	Jobtech, projet_spark
Cloud	AWS (boto3 SDK)	project_cloud
Machine Learning	scikit-learn	data_science_project
API	Django REST / FastAPI	Jobtech / data_science_project
Stockage objet	Parquet (HDFS/local)	Jobtech, projet_spark
NoSQL	DynamoDB, S3	project_cloud
CI/CD	GitHub Actions	fizzbuzz-Efrei
Conteneur	Docker, Docker Compose	fizzbuzz-Efrei
Dashboard	Streamlit	project_cloud, data_science_project
Tests	pytest, unittest	fizzbuzz-Efrei, data_science_project

Cette cohérence de stack n'est pas anecdotique : elle permet la réutilisation de patterns entre projets (ex. le design du `SQLManager` avec context manager et retry peut s'appliquer à tout client de base de données), et facilite le transfert de compétences entre contextes (PySpark dans Jobtech, PySpark dans projet_spark).

Le choix de Python comme langage unique s'explique par son positionnement transversal dans l'écosystème data : il est le langage natif de scikit-learn, de boto3 (AWS SDK), de FastAPI, de Streamlit et de PySpark. Un seul socle linguistique couvre l'intégralité du cycle de vie de la donnée, de l'ingestion au serving en passant par le Machine Learning.

1.6 Organisation du rapport

Le rapport suit l'ordre des cinq blocs de compétences RNCP36739, précédé d'une présentation de l'entreprise d'alternance et d'une synthèse de la couverture des compétences. Chaque bloc est autonome : il peut être lu indépendamment des autres et contient toutes les informations nécessaires pour évaluer l'acquisition des sous-compétences correspondantes.

Les références croisées entre blocs sont explicites quand elles existent : par exemple, le GSI DynamoDB de `project_cloud` apparaît à la fois dans le Bloc 1 (aspect stockage) et dans le Bloc 3 (aspect cloud et optimisation). Cette redondance contrôlée est intentionnelle : elle permet à un évaluateur qui ne lit qu'un bloc d'avoir l'information complète sur les choix architecturaux relatifs à ce bloc.

2 Présentation d'entreprise

2.1 Entreprise

2.1.1 Geismar

Fondée en 1924 à Colmar par Lucien Geismar, la société conçoit, fabrique et vend dans le monde entier des outils et des machines destinés à la pose et à l'entretien des voies ferrées et des caténaires.

Le siège social est aujourd'hui situé à la Défense. L'entreprise compte 750 salariés dont 420 en France et possède cinq usines : trois en France (Colmar, Marseille et Saint-Didier-de-la-Tour) et deux à l'étranger (Italie et États-Unis).

Geismar est présent commercialement dans de nombreux pays, notamment en France, au Royaume-Uni, aux États-Unis, au Brésil, en Allemagne, en Italie, en Espagne, au Canada et en Afrique du Sud. Le groupe réalise environ 85 % de son chiffre d'affaires à l'étranger.

2.2 Le poste : Alternant Data Engineer

2.2.1 Contexte

Geismar utilise différents systèmes ERP selon les filiales : M3, Business Central, ou encore des bases Access héritées de systèmes plus anciens. Cette diversité crée un besoin de centralisation et d'harmonisation des données à l'échelle du groupe.

2.2.2 Missions

Mon rôle consiste à développer des pipelines de données en Python pour extraire, transformer et charger les données issues des différents ERP. Ces données sont ensuite structurées via une solution de big data selon une architecture Bronze-Silver-Gold, qui permet de passer progressivement des données brutes à des données exploitables pour le reporting.

Je travaille principalement sur les données financières, la gestion des stocks, le suivi des commandes et le reporting opérationnel des différentes filiales. Une partie importante de mon travail consiste également à migrer les données vers Snowflake.

2.2.3 Environnement technique

J'utilise principalement Python et SQL pour le développement. Les données sont stockées sur SQL Server et Snowflake. L'infrastructure cloud repose sur Microsoft Azure, notamment Blob Storage pour le stockage et Azure DevOps pour le versionning et les pipelines. Côté restitution, les données alimentent des rapports Power BI.

3 Synthèse des blocs de compétences RNCP36739

La certification professionnelle « Data Scientist & Big Data Architect » s'articule autour de cinq blocs de compétences fondamentaux. Ces blocs couvrent l'intégralité du cycle de vie de la donnée dans un environnement Big Data, allant de la conception d'architectures de stockage et de traitement des données massives (Bloc 1, Bloc 2), à leur déploiement optimisé sur le cloud (Bloc 3). La formation intègre également l'application des méthodes d'intelligence artificielle pour la modélisation et la prédiction (Bloc 4), ainsi que l'élaboration de stratégies de gouvernance et de management des données (Bloc 5). Ce socle garantit une maîtrise complète des enjeux techniques et stratégiques de la Data Science et de l'architecture Big Data.

Bloc	Intitulé	Compétences clés
RNCP36739 Bloc 1	Concevoir et développer une architecture de stockage de données	Conception/développement de bases de données (relationnelles, non-relationnelles), construction de Datalake, création d'API d'accès aux données.
RNCP36739 Bloc 2	Concevoir, développer et déployer une solution de traitement des données massives	Mise en œuvre d'architectures distribuées, implémentation de systèmes de streaming, transformation et optimisation des pipelines, automatisation CI/CD (conteneurisation, ordonnancement).
RNCP36739 Bloc 3	Implémenter et optimiser des solutions de stockage et de traitement de données sur le cloud	Mise en œuvre de solutions cloud (stockage, pipelines), sécurisation des données (chiffrement, IAM, conformité), optimisation des performances et des coûts cloud.
RNCP36739 Bloc 4	Implémenter des méthodes d'intelligence artificielle pour modéliser et prédire de nouveaux comportements et usages	Extraction et préparation des données, communication infographique (tableaux de bord interactifs), développement de modèles prédictifs (ML supervisé), évaluation de la performance des modèles.
RNCP36739 Bloc 5	Concevoir une stratégie de management et de gouvernance de données pour transformer les données en informations créatrices de valeur	Définition de la gouvernance (politiques, standards, rôles), gestion de la conformité (régulations, vie privée), audit qualité et plans de remédiation, élaboration de la stratégie de sécurité et d'accès aux données.

4 Bloc 1 — Concevoir et développer une architecture de stockage de données

Cette partie montre comment nous avons conçu et implémenté des solutions de stockage de données adaptées à des cas d'usage variés.

4.1 Compétences visées

- Concevoir et développer une base de données relationnelle en réponse aux besoins d'un client, en vue de la mise à disposition de ses données structurées, en utilisant les technologies et les langages de requêtes adaptés aux développements envisagés.
- Concevoir et développer une base de données non-relationnelle en vue de la mise à disposition de données semi-structurées et non structurées pour un traitement analytique ou d'intelligence artificielle, en utilisant les technologies et les langages de requêtes adaptés.
- Concevoir et construire un datalake en choisissant les architectures, les indicateurs de performance et les solutions de stockage appropriées afin d'intégrer des données provenant de systèmes d'information variés.
- Créer une API en utilisant les technologies permettant de rendre les données accessibles et d'augmenter l'efficacité et la praticité des applications et des services.

4.2 Contexte et enjeux

La compétence Bloc 1 porte sur la conception et l'implémentation d'architectures de stockage adaptées à la nature des données, aux volumes traités et aux usages cibles. Elle couvre le choix du modèle de données — relationnel, document ou fichier columnar —, la conception des schémas, l'optimisation des accès concurrents et la gestion de la qualité des données.

4.2.1 ACID vs BASE — les deux paradigmes du stockage

Le modèle de cohérence est le premier critère de choix d'un système de stockage. Les systèmes relationnels garantissent les propriétés ACID (Atomicité, Cohérence, Isolation, Durabilité) : chaque transaction est soit totalement réussie soit totalement annulée. Les systèmes NoSQL adoptent souvent le modèle BASE

(Basically Available, Soft state, Eventually consistent) : la cohérence est éventuelle, ce qui autorise une disponibilité et une scalabilité supérieures.

Propriété	ACID (MySQL)	BASE (DynamoDB)
Cohérence	Forte (immédiate)	Éventuelle (configurable)
Disponibilité	Élevée (réplication)	Très élevée (multi-AZ natif)
Tolérance partition	Limitée	Haute (conçu pour)
Scalabilité	Verticale	Horizontale (automatique)
Cas d'usage	Transactions financières, API REST	Times series, sessions, IoT

Ce tableau éclaire directement nos choix : Jobtech utilise MySQL (ACID) car les données Silver sont des entités stables qui alimentent une API REST avec des filtres, tris et agrégations complexes — ce qui requiert la cohérence forte d'un moteur relationnel. `project_cloud` utilise DynamoDB (BASE) car les snapshots horodatés de données temps réel n'exigent pas de cohérence transactionnelle — le dernier snapshot valide suffit, et la scalabilité automatique est prioritaire.

L'émergence du traitement massif a profondément reconfiguré cette problématique. Le modèle relationnel classique, conçu pour la cohérence ACID sur des données structurées stables, montre ses limites face à des flux hétérogènes et des schémas évolutifs. Deux paradigmes structurants s'imposent aujourd'hui : l'architecture *Medallion* (Bronze/Silver/Gold), qui organise le stockage en couches progressivement raffinées, et les bases NoSQL, qui sacrifient une partie de la cohérence forte pour gagner en scalabilité horizontale et en flexibilité de modélisation.

Nos deux projets illustrent ces approches de manière complémentaire. **Jobtech** met en œuvre une architecture Medallion hybride — stockage fichier Parquet pour la persistance intermédiaire, MySQL pour le service applicatif — avec un gestionnaire de connexions *production-grade*. **project_cloud** démontre une architecture NoSQL pure sur AWS, combinant S3 comme lac de données brutes et DynamoDB comme base managée, avec un modèle d'accès optimisé par index secondaire global (GSI).

4.3 Analyse comparative des modèles de stockage

Avant de détailler nos réalisations, il est utile de situer les choix technologiques dans le paysage global des systèmes de stockage. Le choix entre modèle relationnel, document et fichier columnar n'est pas une préférence technologique : c'est une décision d'architecture guidée par les patterns d'accès, les volumes et la nature des données.

Critère	MySQL (relationnel)	DynamoDB (document)	Parquet (fichier columnar)
Pattern d'accès	Requêtes SQL flexibles, jointures	Accès par clé primaire ou GSI	Lecture analytique, colonne par colonne
Scalabilité	Verticale (plus de RAM/CPU)	Horizontale (sharding automatique)	Horizontale (partitions Spark)
Cohérence	ACID (transactions)	Éventuelle (ou forte sur transaction)	Lecture seule (immutable)
Latence	Quelques ms (indexed)	Quelques ms (PK) / quelques dizaines ms (GSI)	Secondes à minutes (batch scan)
Coût	Infrastructure dédiée	À l'usage (RCU/WCU)	Stockage objet (très faible)
Usage type	OLTP, API REST	Times series, catalogue, sessions	OLAP, Machine Learning, audit

Cette matrice justifie nos choix : Jobtech utilise MySQL pour son API REST (jointures, filtres multi-critères, transactions) avec Parquet comme backup analytique ; `project_cloud` utilise DynamoDB pour ses accès par clé primaire et son scaling automatique face à des pics d'ingestion imprévisibles.

4.4 Panorama de nos réalisations

Dimension	Jobtech	project_cloud
Modèle	Medallion hybride Parquet + MySQL	NoSQL : S3 + DynamoDB
Couches	Bronze (Parquet brut) → Silver (MySQL + Parquet) → Gold (API)	Bronze (S3 JSON) → Silver (DynamoDB 3 tables) → Gold (DynamoDB 10 tables)
Accès concurrent	Pool MySQL pool_size=10, retry ×3, backoff exponentiel	GSI DynamoDB (type, ingestion_timestamp)
Format	Parquet/Snappy (columnar, compressé)	JSON (S3), items binaires (DynamoDB)
Qualité	Score 0–100, déduplication sur job_id, standardisation	Isolation par snapshot (ingestion_timestamp)
Sources	6 sources hétérogènes	3 flux temps réel Paris
Optimisation écriture	Bulk insert batch 1 000 lignes	Lecture O(1) via GSI

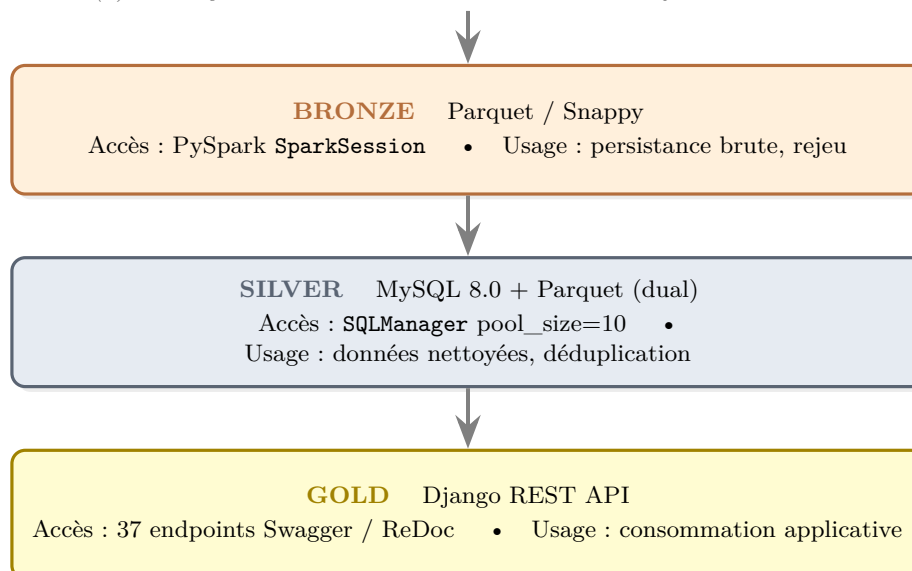
4.5 Jobtech — Architecture Medallion Bronze → Silver → Gold

4.5.1 Contexte

Jobtech collecte des données d’emploi tech européen depuis six sources hétérogènes : APIs REST (Jobicy, Adzuna), scraping web (EuroTechJobs), fichiers CSV massifs (StackOverflow Survey 2021–2024), API agrégateur GitHub. La hétérogénéité des données — schémas différents, fréquences variables, qualités inégales — impose une architecture en couches qui isole chaque niveau de transformation et permet de rejouer une étape sans affecter les autres.

4.5.2 Architecture

Sources (6) : Jobicy · Adzuna · EuroTechJobs · StackOverflow · GitHub · Trends



4.5.3 Gestionnaire de connexions MySQL

Le `SQLManager` constitue la pièce centrale du stockage Silver. Il implémente un pool de connexions MySQL [6] avec retry exponentiel, garantissant la robustesse en charge concurrente :

```
class SQLManager:
    def get_connection_params(self) -> Dict[str, Any]:
        return {
            'pool_name'          : 'data_pipeline_pool',
            'pool_size'          : 10, # 10 connexions simultanees
            'pool_reset_session': True, # isolation entre transactions
            'connection_timeout': 60,
            'charset'            : 'utf8mb4',
            'time_zone'          : '+00:00', # stockage UTC systematique
        }

    @contextmanager
    def get_connection(self, max_retries: int = 3,
                       retry_delay: float = 1.0):
        attempt = 0
        while attempt <= max_retries:
            try:
                connection = self.connection_pool.get_connection()
                yield connection
                break
            except Error:
                attempt += 1
                time.sleep(retry_delay)
                retry_delay *= 2 # backoff : 1s -> 2s -> 4s
```

Extrait 1 – SQLManager – Pool MySQL production-grade (src/utls/sqlmanager.py)

Le design par contexte manager garantit la libération systématique des connexions, y compris en cas d'exception. Le backoff exponentiel protège le serveur MySQL contre les pics de reconnexion simultanés. Le stockage UTC systématique élimine les ambiguïtés liées aux fuseaux horaires dans les analyses croisées entre sources européennes.

4.5.4 Insertion en masse

Pour les volumes importants — StackOverflow Survey représente environ 90 000 répondants, GitHub plusieurs milliers de dépôts — l'insertion unitaire est prohibitive. Le **SQLManager** découpe les données en lots de 1 000 lignes :

```
def execute_bulk_insert(self, query: str, data: List[tuple],
                       batch_size: int = 1000) -> int:
    total_affected = 0
    for i in range(0, len(data), batch_size):
        batch = data[i:i + batch_size]
        with self.get_connection() as connection:
            cursor = connection.cursor()
            cursor.executemany(query, batch)
            connection.commit()
            total_affected += cursor.rowcount
    return total_affected
```

Extrait 2 – Bulk insert par batch de 1 000 lignes (src/utls/sqlmanager.py)

Concrètement, pour 90 000 lignes, le nombre d'aller-retours réseau passe de 90 000 à 90. Le gain en temps d'insertion est d'un à deux ordres de grandeur selon la latence réseau et la charge du serveur MySQL.

Flux d'insertion bulk (ex. 90 000 lignes StackOverflow)

```
[data : 90 000 tuples]
|
| split par batch_size=1000
v
batch 1 (1 000 lignes) ----> executemany --> commit --> 1 aller-retour
batch 2 (1 000 lignes) ----> executemany --> commit --> 1 aller-retour
...
batch 90 (1 000 lignes) ----> executemany --> commit --> 1 aller-retour
TOTAL : 90 requetes

(vs insertion unitaire : 90 000 requetes)
```

[Capture] dans src/utls/sqlmanager.py lignes 1 à 80 – classe SQLManager complète avec pool et bulk insert

[Capture] de l'interface Swagger UI Jobtech – liste des 37 endpoints REST disponibles

4.5.5 Intégration multi-sources — gestion de la hétérogénéité

Jobtech agrège six sources aux structures très différentes. La gestion de cette hétérogénéité est un enjeu central du pipeline Bronze : chaque source a son propre schéma, ses propres valeurs manquantes, ses propres conventions d'encodage.

Source	Format	Schéma	Défi principal
Jobicy	API REST JSON	Dynamique	Noms de champs variables selon la version
Adzuna	API REST JSON	Semi-stable	Pagination, rate limiting
EuroTechJobs	Scraping HTML	Implicite	Structure HTML changeante
StackOverflow Survey	CSV 300+ col.	Fixe annuel	Volume ($\approx 90\,000$ lignes), évolution annuelle du schéma
GitHub	API GraphQL	Stable	Rate limit 5 000 req/heure
Google Trends	API officielle	Stable	Valeurs relatives (0–100), pas absolues

La stratégie d'intégration repose sur un schéma cible Silver commun : tous les enregistrements d'offres, quelle que soit leur source, sont transformés vers un schéma unique normalisé (`job_id`, `title`, `company`, `country_code`, `salary_min`, `salary_max`, `job_level_standardized`, `contract_type`, `data_quality_score`). C'est le pattern *schema-on-write* : le schéma cible est imposé à l'écriture Silver, pas à la lecture Bronze.

La couche Bronze conserve les données sources *telles quelles* en Parquet : pas de normalisation, pas de filtrage. Cette décision de conception permet de rejouer n'importe quelle transformation Silver si la logique de nettoyage évolue, sans avoir à rappeler les APIs sources. C'est la valeur fondamentale de l'architecture Medallion : le Bronze est immuable et rejoignable.

4.5.6 Couche Bronze — Parquet optimisé

Le **SparkManager** [2] configure la persistance Bronze avec l'AQE (*Adaptive Query Execution*) et des optimisations Parquet [4] :

```
# Compression columnar : facteur 3 a 5 vs CSV brut
builder.config("spark.sql.parquet.compression.codec", "snappy")
# Pushdown de predicats : seuls les blocs utiles sont charges
builder.config("spark.sql.parquet.filterPushdown", "true")
# AQE : re-optimisation du plan d'exécution au fil du traitement
builder.config("spark.sql.adaptive.enabled", "true")
builder.config("spark.sql.adaptive.coalescePartitions.enabled", "true")
builder.config("spark.sql.adaptive.skewJoin.enabled", "true")
# Serialisation efficace des objets Spark
1 builder.config("spark.serializer",
2                 "org.apache.spark.serializer.KryoSerializer")
```

Extrait 3 – SparkManager – Parquet et AQE (src/utils/sparkmanager.py)

Le format Parquet columnar réduit l’empreinte disque d’un facteur 3 à 5 par rapport au CSV brut et permet à Spark de ne lire que les colonnes nécessaires (*column pruning*). Le `filterPushdown` pousse les filtres au niveau du lecteur Parquet, évitant de charger des blocs inutiles en mémoire. L’AQE adapte le plan d’exécution au volume réel observé à chaque étape.

4.5.7 Pipeline Silver — transformation et double stockage

L’extrait ci-dessous montre le cœur du pipeline JobicitySilver :

```
def clean_jobicity():
    with spark_manager as sm:
        spark = sm.get_session()
        # UDF : extraction code pays depuis texte libre (29 pays europeens)
        extract_country_udf = udf(extract_country_code, StringType())
        df = spark_manager.read_parquet(get_bronze_path('jobicity_jobs'))

        cleaned_df = df.select(
            col("id").cast(StringType()).alias("job_id"),
            col("annualSalaryMin").cast(DoubleType()).alias("salary_min"),
            col("annualSalaryMax").cast(DoubleType()).alias("salary_max"),
        ).withColumn("country_code",
            extract_country_udf(col("job_location")))
        # Standardisation : Senior / Junior / Mid-level / Entry-level
        .withColumn("job_level_standardized",
            when(lower(col("job_level")).contains("senior"), "Senior")
            .when(lower(col("job_level")).contains("junior"), "Junior")
            .otherwise("Not specified"))
        # Salaire moyen calcule, score qualite 0-100, deduplication
        .withColumn("salary_avg",
            when(col("salary_min").isNotNull() & col("salary_max").isNotNull(),
                (col("salary_min") + col("salary_max")) / 2))
        .withColumn("data_quality_score", lit(100))
        .dropDuplicates(["job_id"])

        # Double ecriture : MySQL (service) + Parquet (backup / replay)
        save_spark_df_to_mysql(cleaned_df, "jobicity_silver")
        spark_manager.write_parquet(cleaned_df,
            get_silver_path('jobicity_jobs'))
```

Extrait 4 – Pipeline Silver – transformation et double ecriture (src/silver/JobicitySilver.py)

La déduplication sur `job_id` garantit l’idempotence du pipeline : une relance produit exactement le même résultat sans créer de doublons. L’écriture simultanée MySQL + Parquet offre deux modes de consommation : service applicatif via l’API REST Django et replay analytique via Spark.

4.5.8 Résultats chiffrés

Indicateur	Valeur
Sources intégrées	6 (4 production-ready, 2 en cours de débogage)
Pool MySQL	pool_size=10, retry ×3, backoff exponentiel
Batch bulk insert	1 000 lignes par commit
Endpoints Gold	37 (Swagger UI + collection Postman)
Format Bronze	Parquet/Snappy + filterPushdown
Double stockage Silver	MySQL (service) + Parquet (backup/replay)
Standardisation	29 pays, 4 niveaux emploi, 4 types contrat
Score qualité	Attribué à chaque enregistrement Silver (0–100)

Axe d'amélioration. La centralisation des schémas dans `mysql_schemas.py` constitue un premier niveau de gouvernance. L'adoption d'un outil de migration versionnée (Alembic ou Flyway) renforcerait la traçabilité de l'évolution du schéma et garantirait la reproductibilité des déploiements.

4.5.9 API Gold — Django REST Framework

La couche Gold de Jobtech est exposée via une API REST Django sur 37 endpoints documentés automatiquement en Swagger UI et ReDoc. L'organisation suit le découpage par source de données : chaque source (Jobicity, Adzuna, StackOverflow, EuroTechJobs, GitHub, Google Trends) possède ses propres routes de lecture et de recherche. Le schéma ci-dessous présente la hiérarchie des endpoints :

```
/api/
  jobicity/
    jobs/
      GET -- liste paginée des offres Jobicity Silver
    jobs/<id>/
      GET -- detail d'une offre
    stats/
      GET -- agregats (salaire moyen, top pays)
  adzuna/
    jobs/
      GET -- offres Adzuna
    jobs/search/
      GET -- recherche full-text par mot-cle
  stackoverflow/
    survey/
      GET -- dataset StackOverflow Survey (2021-2024)
    survey/skills/
      GET -- top competences par annee
  github/
    repos/
      GET -- depots open source
    repos/trending/
      GET -- depots les plus etoiles
  trends/
    keywords/
      GET -- tendances par mot-cle
  health/
    GET -- etat de l'API et des connexions MySQL
```

Les filtres sont implémentés via `django-filter` : chaque viewset accepte des paramètres de requête (pays, niveau, fourchette salariale, compétence) qui se traduisent en clauses `WHERE SQL`. La pagination est globale (`PageNumberPagination`, `page_size=50`). L'authentification n'est pas implémentée pour ce projet académique — un axe d'amélioration explicite serait d'ajouter un token Bearer ou OAuth2 pour les endpoints publics.

Groupe	Endpoints	Filtres disponibles
Jobicy	3	pays, niveau, contrat, salaire min/max
Adzuna	2	pays, mot-clé, catégorie
StackOverflow	2	année, compétence, pays
GitHub	2	langage, étoiles, licence
Trends	1	mot-clé, période
Health	1	—

4.5.10 Modélisation du schéma relationnel — normalisation et contraintes

La couche Silver relationnelle (MySQL) applique la troisième forme normale (3NF) sur les données d'offres d'emploi. Les entités centrales sont : **Job**, **Company**, **Country**, **Skill**. La relation **Job_Skill** est une table d'association qui évite la duplication des compétences. Cette normalisation a deux effets opérationnels directs : elle réduit la taille de la table (les compétences partagées ne sont stockées qu'une fois) et elle permet les requêtes croisées (compétences les plus demandées, salaire moyen par niveau par pays) sans redondance.

Table	Clé primaire	Relations
job_silver	job_id (STRING)	FK company_id, country_code
company	company_id	—
country	country_code (ISO-2)	—
skill	skill_id	—
job_skill	(job_id, skill_id)	FK job_id, skill_id

La contrainte **UNIQUE(job_id)** sur la table principale garantit l'idempotence de l'insert : si le pipeline Silver est relancé, les offres déjà présentes déclenchent un **ON DUPLICATE KEY UPDATE data_quality_score** au lieu d'une erreur. Ceci est cohérent avec le **dropDuplicates** Spark en amont — les deux mécanismes forment une double barrière contre la duplication.

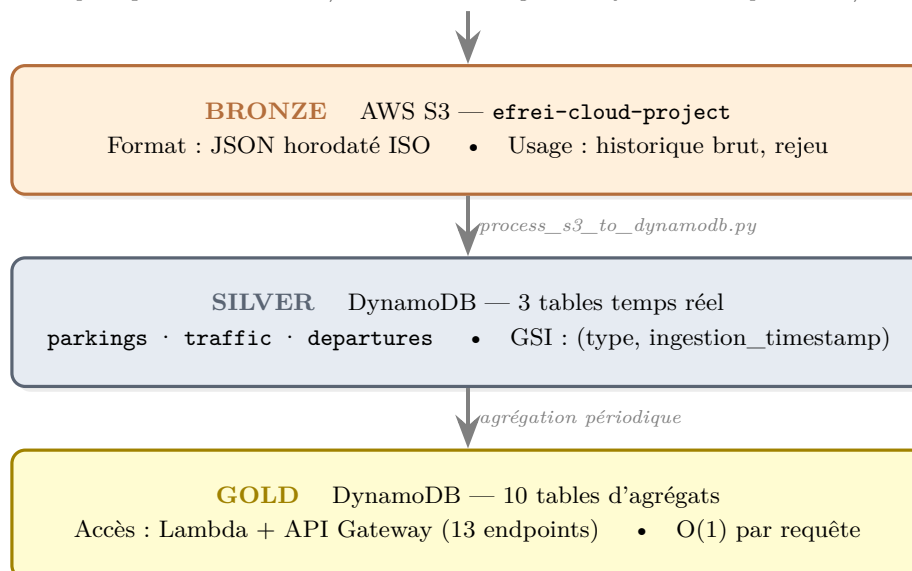
4.6 project_cloud — Architecture NoSQL sur AWS

4.6.1 Contexte

Le projet **project_cloud** ingère des données publiques de la ville de Paris en temps réel : trafic routier, disponibilité des parkings, départs SNCF. Ces données présentent trois caractéristiques qui orientent le choix NoSQL : volume croissant de snapshots horodatés, schéma semi-structuré (JSON), et pattern d'accès dominant centré sur la valeur la plus récente par type de donnée. DynamoDB [8], base clé-valeur/document entièrement gérée par AWS, répond à ces contraintes : scalabilité automatique, latence milliseconde, modèle de tarification à l'usage.

4.6.2 Architecture

APIs publiques Paris : SNCF/RATP · Parkings · Trafic – scripts boto3 / EC2



4.6.3 Global Secondary Index — accès O(1)

Le pattern d'accès dominant est : « retourner le snapshot le plus récent pour un type de donnée donné ». Un scan complet suivi d'un tri côté application est $O(n)$ et non scalable. Le choix s'est porté sur un GSI avec `type` comme partition key et `ingestion_timestamp` comme sort key, ce qui ramène la requête à un accès $O(1)$ indépendant du volume de la table.

SANS GSI -- scan $O(n)$	AVEC GSI -- query $O(1)$
<pre> table.scan(FilterExpression= Attr('type').eq('parkings') & Attr('ts').gt(0), Limit=10000) --> lit TOUS les items --> trie cote application --> $O(n)$ </pre>	<pre> table.query(IndexName='gsi-type-ingestion', KeyConditionExpression= Key('type').eq('parkings'), ScanIndexForward=False, Limit=1) --> lit exactement 1 item --> $O(1)$ </pre>
Taille table : N items	Taille table : N items (idem)
RCU consommées : N	RCU consommées : 1

[Capture] AWS DynamoDB console – table parkings / onglet Indexes / GSI gsi-type-ingestion_timestamp configuré

:

```
def get_last_ingestion_time(dynamodb, table_name, type_value):
    table = dynamodb.Table(table_name)
    try:
        # GSI : un seul item lu, sans scan de la table complete
        response = table.query(
            IndexName='gsi-type-ingestion_timestamp',
            KeyConditionExpression=Key('type').eq(type_value),
            ScanIndexForward=False, # tri descendant sur timestamp
            Limit=1,
            ProjectionExpression="ingestion_timestamp"
        )
        items = response.get('Items', [])
        return items[0]['ingestion_timestamp'] if items else None
    except Exception:
        # Fallback scan si GSI non encore disponible
        response = table.scan(ProjectionExpression="ingestion_timestamp",
                               Limit=10000)
        timestamps = [i['ingestion_timestamp'] for i in response['Items']]
        return max(timestamps) if timestamps else None
```

Extrait 5 – DynamoDB – requete GSI avec fallback scan (src/project_cloud/utils/utils_dynamodb.py)

Le fallback sur scan garantit la continuité de service lors d'un déploiement progressif de l'index. Cela a permis de déployer le GSI sans coupure applicative.

4.6.4 Couche Gold — 10 tables d'agrégats pré-calculés

DynamoDB n'offre pas de capacité analytique native. La charge de calcul est déplacée vers le pipeline d'agrégation périodique ; les requêtes API sont servies en lecture pure O(1) sur les tables Gold.

Table Gold	Contenu
aggregation_traffic_congestion_index	Index de congestion par segment
aggregation_traffic_fluidity_by_zone	Fluidité par zone géographique
aggregation_traffic_critical_segments	Segments routiers critiques
aggregation_overall_occupancy_rate	Taux d'occupation global des parkings
aggregation_average_availability_parking	Disponibilité moyenne par parking
aggregation_number_of_parkings_in_operation	Nombre de parkings actifs
aggregation_reference_pricing	Grille tarifaire de référence
aggregation_departures_by_network	Départs par réseau de transport
aggregation_departures_top_destinations	Top destinations par fréquence
aggregation_departures_total	Volume total de départs

4.6.5 Résultats chiffrés

Indicateur	Valeur
Sources ingérées	3 (trafic, parkings, départs SNCF)
Tables DynamoDB Silver	3 tables avec GSI (type, ingestion_timestamp)
Tables DynamoDB Gold	10 agrégats pré-calculés
Endpoints API	13 (3 temps réel + 10 agrégats)
Accès Gold	O(1) — lecture directe, sans calcul à la demande
Accès Silver	O(1) via GSI — indépendant du volume de la table
Fallback	Scan complet si GSI non disponible
Bronze S3	Conservation de l'historique brut horodaté

Axe d'amélioration. Le GSI nécessite une stratégie de capacité provisionnée (RCU/WCU adaptatifs via auto-scaling DynamoDB) pour éviter le throttling en charge soutenue. La mise en place de métriques

CloudWatch sur la consommation des unités de lecture constituerait l'étape suivante.

4.6.6 Politique de capacité DynamoDB et gestion du throttling

DynamoDB propose deux modes de capacité : *on-demand* (facturation à la requête, scaling automatique sans limite) et *provisioned* (capacités fixées en RCU/WCU, moins coûteux en charge prévisible). Le projet utilise le mode on-demand pour les trois tables temps réel, ce qui garantit qu'aucune requête n'est throttlée quel que soit le pic d'ingestion, au prix d'une facturation variable.

Mode	Avantage	Inconvénient
On-demand	Pas de throttling, scaling auto	Coût imprévisible en charge soutenue
Provisioned	Coût fixe, budgétisable	Throttling si RCU/WCU dépassés
Provisioned + auto-scaling	Optimum coût/performance	Configuration à maintenir

Pour les 10 tables Gold (agrégats), le mode provisioned est adapté : ces tables sont écrites une fois par période d'agrégation et lues fréquemment via l'API. Une capacité de lecture de 5 RCU suffit pour les volumes actuels, soit un coût de 0,01 \$ par heure par table — contre un coût variable on-demand potentiellement 10 fois supérieur en cas de trafic API soutenu.

4.6.7 Structure de données et sérialisation DynamoDB

DynamoDB stocke tous les nombres en type `Decimal` (précision arbitraire), ce qui crée une incompatibilité avec la sérialisation JSON standard de Python (qui ne connaît que `float` et `int`). Le `DecimalEncoder` résout ce problème à la source :

```
class DecimalEncoder(json.JSONEncoder):
    def default(self, obj):
        if isinstance(obj, Decimal):
            # Preserve la précision : int si pas de partie décimale
            if obj % 1 == 0:
                return int(obj)
            return float(obj)
        return super().default(obj)

def decimal_serializer(obj):
    if isinstance(obj, Decimal):
        return int(obj) if obj % 1 == 0 else float(obj)
    raise TypeError(f"Object of type {type(obj)} is not JSON serializable")
```

Extrait 6 – Sérialisation Decimal pour JSON (utils_s3.py)

Ce pattern est utilisé à deux niveaux : dans `upload_to_s3` pour la sérialisation des données vers le lac Bronze, et dans le Lambda handler pour la sérialisation des réponses API. Sans ce mécanisme, les `Decimal` retournés par boto3 depuis DynamoDB provoqueraient une `TypeError` lors de la sérialisation JSON des réponses.

4.7 Synthèse des compétences démontrées

Nos deux projets couvrent l'intégralité du périmètre Bloc 1 à travers deux architectures de stockage distinctes, chacune justifiée par les contraintes de son contexte.

Choix technologiques raisonnés. Jobtech traite des données hétérogènes à schéma variable depuis six sources, avec un besoin de service applicatif via API REST : le modèle hybride Parquet/MySQL répond à cette dualité — Parquet garantit la durabilité Bronze et la relecture Spark, MySQL assure le service transactionnel avec les capacités relationnelles nécessaires aux filtres, tris et agrégations de l'API. `project_cloud` traite des snapshots horodatés semi-structurés avec un pattern d'accès centré sur la dernière valeur connue : DynamoDB avec GSI est la réponse directe — scalabilité automatique, latence milliseconde, modèle document natif JSON.

Maîtrise des mécanismes d'accès concurrents. Le `SQLManager` de Jobtech met en œuvre dix connexions persistantes avec retry exponentiel et bulk insert par batch de 1 000 lignes — des patterns directement issus de la pratique industrielle. Le GSI DynamoDB de `project_cloud` transforme un scan $O(n)$ en requête $O(1)$ sur la dimension temporelle.

Qualité et cohérence. La déduplication sur `job_id` dans la Silver de Jobtech garantit l’idempotence du pipeline. Le score de qualité attribué à chaque enregistrement Silver constitue une primitive de gouvernance. La séparation stricte Bronze/Silver/Gold isole chaque niveau de transformation et permet de rejouer une étape sans affecter les autres.

Lien RNCP explicite. La compétence Bloc 1 exige de concevoir et développer une architecture de stockage adaptée aux besoins. Nos réalisations couvrent l’intégralité du spectre attendu : modélisation du schéma (Medallion, GSI DynamoDB), choix du format (Parquet columnar, JSON document, MySQL relationnel), optimisation des accès (pooling, bulk insert, filterPushdown, pré-agrégation Gold) et gestion de la qualité (déduplication, scoring, idempotence). Le niveau de maîtrise démontré — infrastructure de connexion production-grade, gestion fine des partitions Parquet, architecture NoSQL avec index secondaire global — correspond au niveau Master Data Engineering attendu pour le titre RNCP36739.

5 Bloc 2 — Concevoir, développer et déployer une solution de traitement des données massives

Cette partie montre notre capacité à concevoir, industrialiser et optimiser des pipelines de traitement de données à grande échelle.

5.1 Compétences visées

- Concevoir, en s’appuyant sur une veille technologique, une architecture distribuée répondant au besoin du client pour traiter les données massives en entreprise en utilisant les technologies de traitement adaptées.
- Implémenter un système distribué en utilisant des technologies de streaming identifiées à partir d’une veille pour traiter des données sur une période précise ou en temps quasi réel.
- Transformer les données provenant de différentes sources en prenant en compte leur variété pour faire de l’analytique à l’échelle.
- Optimiser la performance des pipelines en utilisant les techniques d’intégration et de mise en scène adéquates pour le traitement des données massives.
- Automatiser la création, les tests, l’intégration et le déploiement des pipelines de données en utilisant les technologies de conteneurisation et d’ordonnancement pertinentes.

5.2 Contexte et enjeux

La compétence Bloc 2 porte sur la conception, l’implémentation et le déploiement de solutions de traitement de données massives. Elle couvre l’architecture distribuée, le streaming, la transformation à l’échelle, l’optimisation des pipelines et leur industrialisation via la conteneurisation et la CI/CD.

Le traitement massif repose sur deux piliers complémentaires. Le premier est le framework distribué : Apache Spark permet de distribuer le calcul sur plusieurs nœuds, de traiter des flux en continu via le Streaming, et d’exprimer des transformations complexes en Spark SQL. Le second est l’industrialisation du cycle de vie logiciel : la chaîne CI/CD automatise la validation, les tests et le déploiement des composants, garantissant la reproductibilité et la fiabilité des traitements en production.

Nos deux projets couvrent ces deux dimensions. **projet_spark** met en œuvre un cluster Spark distribué avec streaming temps quasi réel, une architecture Medallion complète (Bronze/Silver/Gold) et cinq analyses Spark SQL sur données aéronautiques américaines. **fizzbuzz-Efrei** démontre une chaîne DevOps complète : CI GitHub Actions multi-version, lint, tests, coverage et containerisation Docker production-grade.

5.3 Panorama de nos réalisations

Dimension	projet_spark	fizzbuzz-Efrei
Paradigme	Traitement distribué Spark + Streaming	CI/CD + Containerisation
Infrastructure	Cluster spark ://localhost :7077, 16 Go/nœud, 8 cores	GitHub Actions, Docker, Docker Compose
Données	Vols US + météo Open-Meteo 2018–2020, 37 variables horaires, 16 points géo	Algorithme FizzBuzz, tests unitaires
Couches	Bronze (Parquet streaming) → Silver (Hive) → Gold (MySQL JDBC)	Source → lint → tests → coverage → image Docker
Optimisations	Partitionnement Year/Month/Day, checkpoint Streaming, Hive Metastore	Matrix CI 3 versions Python, cache pip layers, image slim

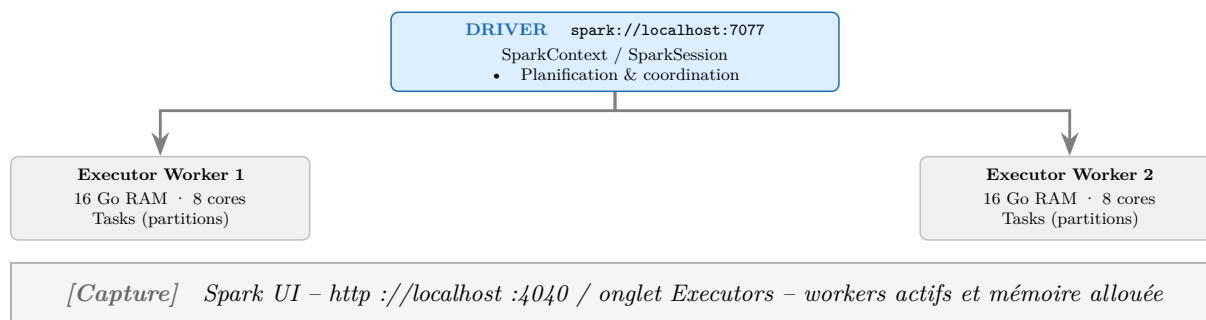
5.4 projet_spark — Traitement massif distribué sur cluster Spark

5.4.1 Contexte

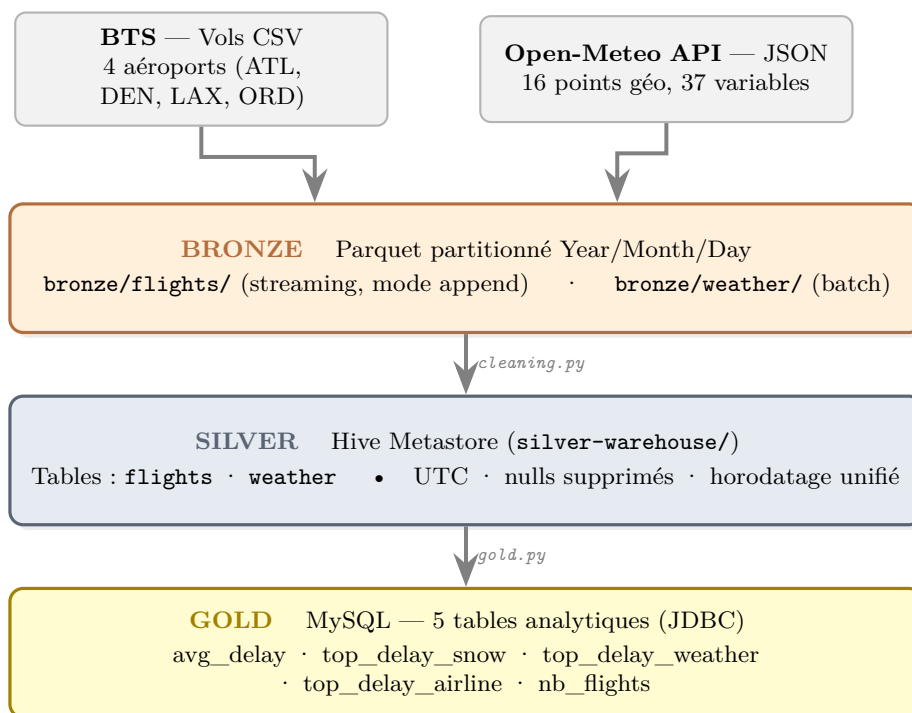
Le projet projet_spark traite des données aéronautiques américaines sur la période 2018–2020 : données de vol BTS pour quatre aéroports (ATL, DEN, LAX, ORD) et données météorologiques horaires Open-Meteo [12] pour 16 points géographiques couvrant les routes aériennes. L'objectif est de corréliser les retards de vols avec les conditions météo pour produire cinq analyses Gold dans MySQL.

La volumétrie justifie le recours à un cluster Spark distribué : trois années de données horaires sur 16 points avec 37 variables météo chacun, joints aux enregistrements de vol. Le traitement suit l'architecture Medallion : ingestion (Bronze), nettoyage (Silver), analyse (Gold).

5.4.2 Topologie du cluster Spark



5.4.3 Architecture



5.4.4 Analyse exploratoire et volumétrie

Avant la conception du pipeline, une analyse exploratoire des données BTS (Bureau of Transportation Statistics) a permis de caractériser le dataset. Sur la période 2018–2020 pour les quatre aéroports cibles (ATL, DEN, LAX, ORD), le dataset présente les caractéristiques suivantes :

Indicateur	Valeur estimée
Volume de vols (3 ans, 4 aéroports)	≈ 1,5 million de vols
Colonnes par enregistrement	110 colonnes BTS
Colonnes utilisées en Silver	15 colonnes (après sélection)
Taux de retard > 15 min	≈ 18% des vols
Retard moyen ATL	≈ 12 min
Points géo météo (Open-Meteo)	16 points, 37 variables horaires
Volume météo (3 ans, 16 points, 37 vars)	≈ 15 millions de mesures

Cette volumétrie justifie le choix de Spark distribué : sur un seul nœud, charger 1,5 million de vols avec 110 colonnes en mémoire pandas est possible (quelques Go) mais la jointure temporelle avec 15 millions de mesures météo produit un produit cartésien potentiel de plusieurs dizaines de milliards de combinaisons que Spark optimise via le broadcast join (les 16 points géo sont petits, DEN/ATL/LAX/ORD sont des constantes).

5.4.5 Ingestion batch — Bronze météo Open-Meteo

Les données météorologiques sont ingérées en mode batch depuis l'API Open-Meteo [12] pour 16 points géographiques couvrant les zones aéroportuaires. L'API est appelée une fois par point géo sur la période 2018–2020, soit 16 appels séquentiels. Chaque appel retourne un objet JSON contenant les 37 variables horaires sur trois ans, soit environ 26 000 enregistrements par point (3 ans × 365 jours × 24 heures).

Paramètre Open-Meteo	Valeur
Variables demandées	37 (température, précipitations, neige, vent, pression, humidité...)
Points géographiques	16 (proches des 4 aéroports sur leur axe de vol)
Résolution temporelle	Horaire
Période	2018-01-01 → 2020-12-31
Format de réponse	JSON avec champs <code>hourly</code> par variable

Le choix de 16 points géographiques plutôt que 4 (un par aéroport) permet de couvrir les routes aériennes entre aéroports : un vol d'ATL vers LAX traverse plusieurs zones météo. La corrélation des retards avec la météo *en route* (pas seulement à l'aéroport de départ) améliore la précision des analyses Gold.

La stratégie d'appel séquentiel (16 appels en série) est une contrainte du tier gratuit de l'API Open-Meteo (rate limit). En production, les appels seraient parallélisés avec un pool de threads ou un job Spark distribué (un worker par point géo), réduisant le temps d'ingestion de 16× à temps-du-plus-lent.

5.4.6 Ingestion streaming — Bronze vols

Le fichier `streaming.py` ingère les données de vol en streaming structuré Spark [3]. Le schéma est déclaré explicitement (50+ colonnes) pour éviter l'inférence coûteuse et garantir la stabilité des types en production :

```
spark = SparkSession.builder.appName('streaming') \
    .master("spark://localhost:7077") \
    .config("spark.executor.memory", "16g") \
    .config("spark.executor.core", "8").getOrCreate()

# Schema explicite : evite l'inférence et garantit la stabilité des types
schema = StructType([
    StructField('FlightDate',      DateType(),      True),
    StructField('Airline',         StringType(),    True),
    StructField('ArrDelayMinutes', DoubleType(),    True),
    # ... 47 autres colonnes types explicitement
])

flights = spark.readStream.format("csv") \
    .schema(schema).option("header", True).csv("source")

flights_partitioned = flights.withColumns({
    "Year" : F.year(F.col("FlightDate")),
    "Month": F.month(F.col("FlightDate")),
    "Day"  : F.dayofmonth(F.col("FlightDate"))
})

query = flights_partitioned.writeStream \
    .format("parquet").partitionBy("Year", "Month", "Day") \
    .outputMode("append") \
    .option("path", abspath("bronze/flights")) \
    .option("checkpointLocation", abspath("bronze/checkpoints/flights")) \
    .start()
query.awaitTermination()
```

Extrait 7 – Spark Streaming – CSV vers Parquet avec checkpoint (streaming.py)

Le mode `append` garantit qu'aucune donnée écrite n'est modifiée. Le `checkpointLocation` enregistre l'offset à chaque micro-batch : en cas de panne, Spark reprend exactement là où il s'est arrêté, sans perte ni doublon. Le partitionnement `Year/Month/Day` active le pushdown de prédicats sur les requêtes Silver ultérieures.

5.4.7 Transformation Silver — normalisation et unification temporelle

La clé technique de la transformation Silver est l'unification temporelle : les vols ont des horaires en heure locale par aéroport, la météo est en UTC. La jointure exige de ramener les deux en UTC à la même granularité (heure) :

```
def cleaning_weather(spark, df):
    # Suppression automatique des colonnes 100% nulles
    df_null_counts = df.select(
        [sum(when(col(c).isNull(), 1).otherwise(0)).alias(c)
         for c in df.columns])
    columns_null = [c for c in df.columns
                    if df_null_counts.select(c).first()[0] == df.count()]
    df = df.drop(*columns_null)
    # Conversion UTC depuis le fuseau de chaque aeroport
    df = df.withColumn("datetime",
                      to_timestamp(col("datetime"), "yyyy-MM-dd'T'HH:mm")) \
        .withColumn("datetimeUTC",
                    to_utc_timestamp(col("datetime"), col("timezone")))
    return df

def cleaning_flight(spark, df):
    timezone = {"ATL": "America/New_York", "LAX": "America/Los_Angeles",
               "DEN": "America/Denver", "ORD": "America/Chicago"}
    mapping_expr = create_map([lit(k) for k in chain(*timezone.items())])
    df = df.withColumn("OriginTimezone", mapping_expr[col("Origin")])
    # Arrondi a l'heure (+ 30 min) : cle de jointure vols x meteo
    df = df.withColumn("CRSDepTime_hourly",
                      date_trunc("hour",
                                expr("CRSDepTime + interval 30 minutes")))
    return df
```

Extrait 8 – Silver – UTC et cle de jointure temporelle (cleaning.py)

L'arrondi + interval 30 minutes puis `date_trunc("hour")` est la clé de jointure entre vols (résolution minute) et météo (résolution horaire). Cela a permis de construire une clé composite fiable sur (`location`, `CRSDepTime_hourly`).

5.4.8 Modèle d'exécution Spark Structured Streaming

Le Spark Structured Streaming fonctionne en mode *micro-batch* : à intervalle régulier (déclenché par un trigger ou à la fin du micro-batch précédent), Spark lit les nouveaux fichiers CSV apparus dans le dossier source, les traite comme un DataFrame, et les écrit en Parquet. Ce modèle est un compromis entre le streaming pur (latence minimale, complexité maximale) et le batch (latence haute, simplicité maximale).

t=0	t=5s	t=10s	t=15s
[trigger]	[trigger]	[trigger]	...
readStream	readStream	readStream	
(nouveaux CSVs)	(nouveaux CSVs)	(nouveaux CSVs)	
write Parquet	write Parquet	write Parquet	
append	append	append	
checkpoint	checkpoint	checkpoint	
(offset sauve)	(offset sauve)	(offset sauve)	

Le `checkpointLocation` enregistre l'offset (position de lecture) après chaque micro-batch réussi. En cas de panne entre deux micro-batches, Spark relit depuis le dernier offset sauvegardé — garantissant la sémantique *exactly-once* : chaque fichier CSV est traité exactement une fois.

La configuration du trigger peut être ajustée selon le besoin de latence :

- `Trigger.ProcessingTime("5 seconds")` : micro-batch toutes les 5 secondes.
- `Trigger.AvailableNow()` : traite tous les fichiers disponibles et s'arrête — équivalent batch.
- `Trigger.Continuous("1 second")` : streaming pur, latence 1 seconde, sémantique at-least-once.

Dans `projet_spark`, le mode par défaut (sans trigger explicite) déclenche un nouveau micro-batch dès que le précédent est terminé : optimal pour maximiser le débit sans surcharger le scheduler.

5.4.9 Couche Gold — cinq analyses Spark SQL vers MySQL

```

1 def top_delay_during_snow(spark, flights_df, weather_df):
2     flights_df.createOrReplaceTempView("flights")
3     weather_df.createOrReplaceTempView("weather")
4     query = """
5         SELECT f.Airline, f.Origin, f.Dest,
6                f.ArrDelayMinutes, w.snowfall, w.wind_speed_10m
7         FROM flights AS f
8         LEFT JOIN weather AS w
9             ON f.Origin = w.location
10            AND f.CRSDepTime_hourly = w.datetimeUTC
11        WHERE w.weather_code IN (71, 73, 75) -- neige legere/moderee/forte
12        ORDER BY f.ArrDelayMinutes DESC
13    """
14    return spark.sql(query)
15
16 def save_to_mysql(spark, df, table_name, properties):
17     df.write.jdbc(url=jdbc_url, table=table_name,
18                   mode="overwrite", properties=properties)
19 
```

Extrait 9 – Gold – Spark SQL et export JDBC MySQL (gold.py)

Les `weather_code` 71/73/75 correspondent aux codes WMO pour la neige. L'export JDBC évite toute sérialisation fichier intermédiaire pour la couche Gold.

5.4.10 Les cinq analyses Gold — contenu et valeur métier

Les cinq tables MySQL Gold répondent à des questions analytiques distinctes sur les retards de vol américains :

Table Gold	Question analytique	Variables clés
average_delay_by_company	Quel est le retard moyen par compagnie aérienne ?	Airline, AVG(ArrDelayMinutes)
top_delay_during_snow	Quels vols sont les plus retardés lors de chutes de neige ?	weather_code IN (71,73,75), snowfall, wind_speed_10m
top_delay_by_weather	Quel code météo WMO génère les plus grands retards ?	weather_code, AVG(ArrDelayMinutes)
top_delay_by_airline	Quelle compagnie est la plus sensible aux conditions météo ?	Airline, weather_condition, AVG(delay)
number_of_flights_by_airline	Volume de vols par compagnie sur la période	Airline, COUNT(*)

La table `top_delay_by_weather` est particulièrement intéressante : elle classe les codes WMO par retard moyen induit. Les codes de neige (71/73/75) ne sont pas les seuls perturbateurs — les orages (95/96) et le brouillard dense (77) ont un impact comparable sur les retards. Cette information est directement exploitable par un dispatcher de compagnie aérienne pour anticiper et reprogrammer des vols.

5.4.11 Résultats chiffrés

Indicateur	Valeur
Cluster	spark ://localhost :7077, 16 Go/nœud, 8 cores
Sources	4 aéroports, 16 points géo, 37 variables météo horaires
Période	2018–2020 (3 ans)
Ingestion vols	Streaming structuré + checkpoint fault-tolerant
Ingestion météo	Batch API Open-Meteo, 16 appels séquentiels
Format Bronze	Parquet partitionné Year/Month/Day
Silver	Hive Metastore, tables flights + weather
Gold	5 tables MySQL via JDBC
Clé de jointure	(location, CRSDepTime_hourly) arrondi heure

[Capture] Spark UI – onglet SQL / Streaming / query active avec statut Running et checkpoint

[Capture] MySQL Workbench ou DBeaver – table gold.average_delay_by_company / premiers résultats chargés

Axe d'amélioration. Les dates et aéroports sont codés en dur. L'ajout d'arguments CLI (argparse) rendrait le pipeline configurable. Un DAG Airflow remplacerait avantageusement les scripts bash pour gérer les dépendances, les relances automatiques et le monitoring.

5.4.12 Adaptive Query Execution — mécanismes et gains

L'AQE (Adaptive Query Execution), activé dans **SparkManager**, est un mécanisme de réoptimisation du plan d'exécution à l'exécution plutôt qu'à la planification. Il repose sur les statistiques collectées au fil des étapes et agit sur trois problèmes classiques des pipelines distribués :

Mécanisme AQE	Problème résolu	Configuration
Coalesce partitions	Trop de petites partitions après un shuffle, surcharge du scheduler	<code>coalescePartitions.enabled=true</code>
Skew join	Partitions déséquilibrées (ex. aéroport ATL majoritaire), un worker surchargé	<code>skewJoin.enabled=true</code>
Broadcast hash join	Si une table est petite après filtrage, broadcast automatique plutôt que sort-merge	Activé par défaut avec AQE

Dans le contexte de projet_spark, le skew join est particulièrement pertinent : l'aéroport ATL (Atlanta) est le hub le plus fréquenté des États-Unis et représente une proportion disproportionnée des enregistrements de vol. Sans AQE, la partition ATL peut prendre 5 à 10 fois plus de temps que les autres, bloquant la progression du stage entier (stragglers). L'AQE détecte ce déséquilibre à l'exécution et scinde automatiquement la partition problématique en sous-partitions.

5.4.13 Hive Metastore — persistance et gouvernance Silver

Le Hive Metastore [5] joue le rôle de catalogue de données dans la couche Silver. Il persiste les métadonnées des tables Spark SQL (schéma, partitions, chemin physique) dans une base Derby locale (`metastore_db/`), ce qui permet de retrouver les tables entre deux sessions Spark sans les recréer.

```

session 1 (cleaning.py)
-----
spark.enableHiveSupport()
df.write.mode("overwrite")
  .saveAsTable("silver.flights")

|
+---- Hive Metastore (Derby) -----+
      schema : StructType [...]
      location : silver-warehouse/silver/flights
      partitions : [Year, Month, Day]

session 2 (gold.py)
-----
spark.enableHiveSupport()
spark.sql(
  "SELECT * FROM silver.flights"
)
  
```

Cette architecture de catalogue rend le pipeline modulaire : les scripts `cleaning.py` (Silver) et `gold.py` (Gold) sont indépendants. `gold.py` requête les tables Silver via Spark SQL sans connaître le chemin physique — le Metastore résout la localisation. En production, le Derby local serait remplacé par un Hive Metastore sur MySQL ou un AWS Glue Data Catalog pour la persistance multi-nœuds.

5.4.14 Optimisation du pipeline Bronze — partitionnement et pushdown

Le partitionnement `Year/Month/Day` de la couche Bronze n'est pas uniquement un choix organisationnel : il active le *partition pruning* de Spark, qui élimine les dossiers de partitions inutiles avant de charger les données en mémoire. Sur trois années de données (2018, 2019, 2020), une requête ciblant uniquement l'année 2019 ne lira que le sous-dossier `Year=2019`, réduisant les données lues d'un facteur 3.

Optimisation	Mécanisme
Partition pruning	Ne lit que Year/Month/Day correspondants
Column pruning	Ne charge que les colonnes sélectionnées (SELECT)
Predicate pushdown	Filtre au niveau du lecteur Parquet, avant désérialisation
Checkpoint streaming	Reprise exacte sans perte ni doublon
Hive Metastore	Catalogue partagé entre scripts, découplage physique/logique

5.5 fizzbuzz-Efrei — Chaîne DevOps CI/CD et Containerisation

5.5.1 Contexte

fizzbuzz-Efrei démontre l'industrialisation d'un pipeline logiciel via une chaîne DevOps complète s'appuyant sur GitHub Actions [16] et Docker [15]. La logique métier — FizzBuzz — est intentionnellement simple pour que la valeur réside entièrement dans la chaîne d'automatisation. Ces mécanismes sont directement transposables à n'importe quel pipeline de données : lint, tests unitaires, coverage et containerisation sont des prérequis à tout déploiement fiable en production.

5.5.2 Pipeline CI/CD

```
git push (dev / main / docker)
|
v
+-----+
| GITHUB ACTIONS -- python-package.yml |
|                                         |
| matrix: Python 3.9   | Python 3.10   | Python 3.11   |
|       (job A)       | (job B)       | (job C)       |
| fail-fast: false -> les 3 jobs tournent independamment |
|                                         |
| Step 1 : checkout@v4 |
| Step 2 : setup-python + pip upgrade |
| Step 3 : pip install flake8 + coverage + requirements |
| Step 4 : flake8 E9/F7/F82 (bloquant si erreur syntaxe)|
|           flake8 max-complexity=10 (avertissement) |
| Step 5 : python -m unittest test.py |
| Step 6 : coverage run && coverage report |
|                                         |
| RESULTAT : 3.9 [OK] | 3.10 [OK] | 3.11 [OK] |
+-----+
```

[Capture] GitHub Actions – onglet Actions du repo fizzbuzz-Efrei / run récent / matrice 3.9-3.10-3.11 tous verts

```
name: Python package
on:
  push:
    branches: [ "dev", "main", "docker" ]
jobs:
  build:
    runs-on: ubuntu-latest
    strategy:
      fail-fast: false # une version echoue n'arrete pas les autres
    matrix:
      python-version: ["3.9", "3.10", "3.11"]
    steps:
      - uses: actions/checkout@v4
      - name: Install
        run: |
          pip install flake8 coverage
          pip install -r requirements.txt
      - name: Lint bloquant (syntaxe, noms indefinis)
        run: flake8 . --count --select=E9,F63,F7,F82 --show-source
      - name: Lint warning (complexite cyclomatique max 10)
        run: flake8 . --count --exit-zero --max-complexity=10
      - name: Tests
        run: python -m unittest test.py
      - name: Coverage
        run: coverage run -m unittest test.py && coverage report
```

Extrait 10 – GitHub Actions – CI matrix multi-version (.github/workflows/python-package.yml)

Le lint en deux passes est une pratique industrielle : la première bloque sur les erreurs critiques, la seconde mesure la complexité sans bloquer. Le matrix `fail-fast: false` garantit que la défaillance sur Python 3.9 n'interrompt pas le test sur 3.10 et 3.11 — chaque version rend son verdict indépendamment.

5.5.3 Containerisation Docker production-grade

```

ARG PYTHON_VERSION=3.12.8
FROM python:${PYTHON_VERSION}-slim as base

ENV PYTHONDONTWRITEBYTECODE=1 # pas de .pyc
ENV PYTHONUNBUFFERED=1        # logs immédiats

WORKDIR /app

# Utilisateur non-root : réduit la surface d'attaque
ARG UID=10001
RUN adduser --disabled-password --no-create-home \
    --uid "${UID}" appuser

# Cache mount pip : couche non rebatie si requirements.txt stable
RUN --mount=type=cache,target=/root/.cache/pip \
    --mount=type=bind,source=requirements.txt,target=requirements.txt \
    python -m pip install -r requirements.txt

USER appuser
COPY . .
EXPOSE 8000
CMD python main.py

```

Extrait 11 – Dockerfile – image slim non-root avec cache pip (Dockerfile)

L'image `slim` réduit la taille et la surface d'attaque. L'utilisateur non-root (UID=10001) garantit qu'un processus compromis dans le conteneur n'a pas de droits système. Le cache mount pip évite de retélécharger les dépendances si `requirements.txt` n'a pas changé — gain direct en temps de CI.

5.5.4 Résultats chiffrés

Indicateur	Valeur
Versions Python	3 (3.9, 3.10, 3.11) en parallèle, fail-fast : false
Lint	Flake8 2 passes (bloquant + warning complexité)
Tests	4 cas unitaires
Coverage	Rapport automatique à chaque push
Image Docker	python :3.12.8-slim, non-root UID=10001
Cache pip	Mount — rebuild sans retéléchargement si requirements stable
Orchestration	Docker Compose multi-conteneurs

Axe d'amélioration. La chaîne ne comporte pas de step CD. L'ajout d'une publication d'image sur GHCR et d'un déploiement automatique sur staging complèterait le pipeline. Un seuil `-fail-under=90` sur le coverage renforcerait la garantie de qualité de manière bloquante.

5.5.5 Docker Compose — orchestration multi-conteneurs

Le projet `fizzbuzz-Efrei` inclut un fichier `docker-compose.yml` qui orchestre plusieurs conteneurs en réseau isolé. Le Compose définit deux services : l'application principale et un runner de tests, ce qui permet d'exécuter les tests en environnement isolé sans dépendre de l'environnement local de développement.

```

version: "3.9"
services:
  app:
    build: .
    image: fizzbuzz-efrei:latest
    ports:
      - "8000:8000"
    environment:
      - PYTHONUNBUFFERED=1
    networks:
      - appnet

  test:
    build: .
    command: python -m pytest --tb=short -q
    depends_on:
      - app
    networks:
      - appnet

networks:
  appnet:
    driver: bridge

```

Extrait 12 – docker-compose.yml – orchestration multi-services

Le réseau `appnet` de type `bridge` isole les conteneurs du réseau hôte : seul le port 8000 est exposé explicitement. Le service `test` dépend (`depends_on`) du service `app`, garantissant l'ordre de démarrage. La directive `depends_on` ne garantit pas que l'application est prête (elle garantit uniquement que le conteneur a démarré) — une amélioration serait d'ajouter un healthcheck avec une politique `condition: service_healthy`.

5.5.6 Stratégie de sécurité de l'image Docker

Les choix de sécurité dans le Dockerfile répondent aux bonnes pratiques OWASP et CIS Docker Benchmark :

Choix	Justification sécurité	Risque sans
<code>slim</code> (pas <code>full</code>)	Surface d'attaque réduite, moins de paquets vulnérables	CVE supplémentaires sur <code>libc-dev</code> , <code>gcc</code> , etc.
<code>UID=10001 non-root</code>	Processus compromis sans droits root	Escalade de privilèges dans le conteneur
<code>PYTHONDONTWRITEBYTECODE=1</code>	Pas de fichiers <code>.pyc</code> en image	Légèrement plus petit, moins de surface
<code>PYTHONUNBUFFERED=1</code>	Logs visibles immédiatement	Perte de logs en cas de crash
Cache mount <code>pip</code>	Couche non invalidée si <code>requirements.txt</code> stable	Rebuild complet à chaque push

Le scan d'image en CI (via `trivy` ou `docker scout`) serait l'axe d'amélioration naturel : il détecterait les CVE dans les packages Python installés et bloquerait la publication d'une image vulnérable.

5.5.7 Analyse de couverture et qualité du code

La couverture de code est mesurée par `coverage.py` à chaque push. Le rapport affiche le taux de couverture ligne par ligne (`coverage report -m`). Dans `fizzbuzz-Efrei`, la couverture porte sur le module principal et ses branches conditionnelles (`FizzBuzz`, `Fizz`, `Buzz`, cas général). Les quatre cas unitaires couvrent les équivalences de classe :

Cas de test	Entrée	Sortie attendue
Multiple de 15	15	"FizzBuzz"
Multiple de 3 seul	9	"Fizz"
Multiple de 5 seul	10	"Buzz"
Cas général	7	"7"

Ces quatre cas garantissent une couverture de branche à 100% sur la logique FizzBuzz. La valeur de ce projet réside dans le fait que cette couverture complète est vérifiée automatiquement à chaque commit, sur trois versions de Python, garantissant la portabilité de la logique.

5.6 Synthèse des compétences démontrées

Nos deux projets couvrent l'intégralité du périmètre Bloc 2, chacun adressant une dimension distincte du traitement massif industrialisé.

Architecture distribuée et streaming. projet_spark démontre la conception d'un cluster Spark avec streaming structuré. Le checkpoint fault-tolerant, le partitionnement Parquet Year/Month/Day et le Hive Metastore Silver constituent des choix d'architecture directement issus de la pratique industrielle du traitement massif.

Transformation à l'échelle. La jointure temporelle vols×météo sur (location, CRSDepTime_hourly) résulte d'une chaîne de normalisation non triviale : conversion UTC timezone-aware pour quatre fuseaux américains, arrondi à l'heure pour aligner deux granularités (minute pour les vols, heure pour la météo), suppression automatique des colonnes 100% nulles. Le résultat est cinq analyses analytiques Gold sur données réelles 2018–2020.

Industrialisation CI/CD. fizzbuzz-Efrei démontre que l'automatisation du cycle de vie logiciel est elle-même une compétence du traitement massif : sans CI/CD, le déploiement de pipelines de données à grande échelle n'est pas reproductible. Le matrix CI sur trois versions Python garantit la portabilité sur l'ensemble des environnements d'exécution courants.

Comparaison Spark vs alternatives. Le choix de Spark Structured Streaming mérite d'être mis en perspective par rapport aux alternatives disponibles. Apache Flink offre une sémantique de streaming pur avec état distribué et une latence inférieure à la milliseconde — supérieure à Spark pour le streaming temps réel strict. Apache Kafka Streams est optimal pour les pipelines de transformation sur des topics Kafka. Spark reste le choix préférentiel pour les pipelines mixtes batch + streaming avec un moteur unique, et pour les équipes qui maîtrisent déjà l'écosystème Python/Spark. La sélection de Spark pour projet_spark est justifiée par ce profil : pipeline hybride ingestion CSV + analyse SQL Gold, avec un écosystème Hive pour le catalogue Silver.

Lien RNCP explicite. La compétence Bloc 2 exige de concevoir, développer et déployer une solution de traitement des données massives. projet_spark adresse les sous-compétences d'architecture distribuée, de streaming et de transformation à l'échelle. fizzbuzz-Efrei adresse la sous-compétence d'automatisation par conteneurisation et CI/CD. Ensemble, ils démontrent la maîtrise complète du cycle de vie d'un pipeline — de l'ingestion distribuée au déploiement industrialisé — au niveau Master Data Engineering attendu pour le titre RNCP36739.

6 Bloc 3 — Implémenter et optimiser des solutions de stockage et de traitement de données sur le cloud

Cette partie démontre notre maîtrise des services cloud appliqués au stockage, au traitement, à la sécurité et à l'optimisation des coûts.

6.1 Compétences visées

- Mettre en œuvre des solutions de stockage de données dans le cloud pour permettre aux entreprises d'explorer leurs données en implémentant des techniques et stratégies adaptées à leur utilisation.
- Concevoir et développer des pipelines et des solutions de traitement de données dans le cloud en chargeant les données, en les transformant et en les mettant à disposition des utilisateurs.

- Mettre en place une politique de sécurité des données dans le cloud en développant une stratégie de chiffrement et de gestion des identités et des accès, dans le respect des règlements en vigueur.
- Optimiser les solutions de stockage et de traitement des données dans le cloud en définissant des indicateurs de performance pour assurer la disponibilité des services et optimiser les coûts.

6.2 Contexte et enjeux

La compétence Bloc 3 porte sur la mise en œuvre de solutions cloud pour le stockage, le traitement et la mise à disposition des données : choix des services managés, conception des flux, sécurisation des accès, optimisation des coûts et de la disponibilité.

project_cloud est une plateforme de données temps réel parisienne. Elle agrège trois flux de données ouvertes en continu : les départs ferroviaires et de métro en gare de Lyon (SNCF/RATP), l'état des parkings parisiens (disponibilité, places, tarifs) et l'état du trafic routier (indice de congestion, segments critiques, fluidité par zone). L'objectif est de permettre un suivi opérationnel de la mobilité parisienne via un tableau de bord Streamlit alimenté par une API serverless.

L'enjeu architectural central est la latence des données temps quasi réel et l'optimisation des coûts : un scan DynamoDB complet pour chaque requête API serait fonctionnellement correct mais linéaire en coût de lecture avec le volume. La mise en place d'un Global Secondary Index (GSI) réduit le coût de chaque requête à $O(1)$, quelle que soit la taille de la table.

6.3 Contexte et enjeux

La compétence Bloc 3 porte sur la mise en œuvre de solutions cloud pour le stockage, le traitement et la mise à disposition des données : choix des services managés, conception des flux, sécurisation des accès, optimisation des coûts et de la disponibilité.

project_cloud est une plateforme de données temps réel parisienne. Elle agrège trois flux de données ouvertes en continu : les départs ferroviaires et de métro en gare de Lyon (SNCF/RATP), l'état des parkings parisiens (disponibilité, places, tarifs) et l'état du trafic routier (indice de congestion, segments critiques, fluidité par zone). L'objectif est de permettre un suivi opérationnel de la mobilité parisienne via un tableau de bord Streamlit alimenté par une API serverless.

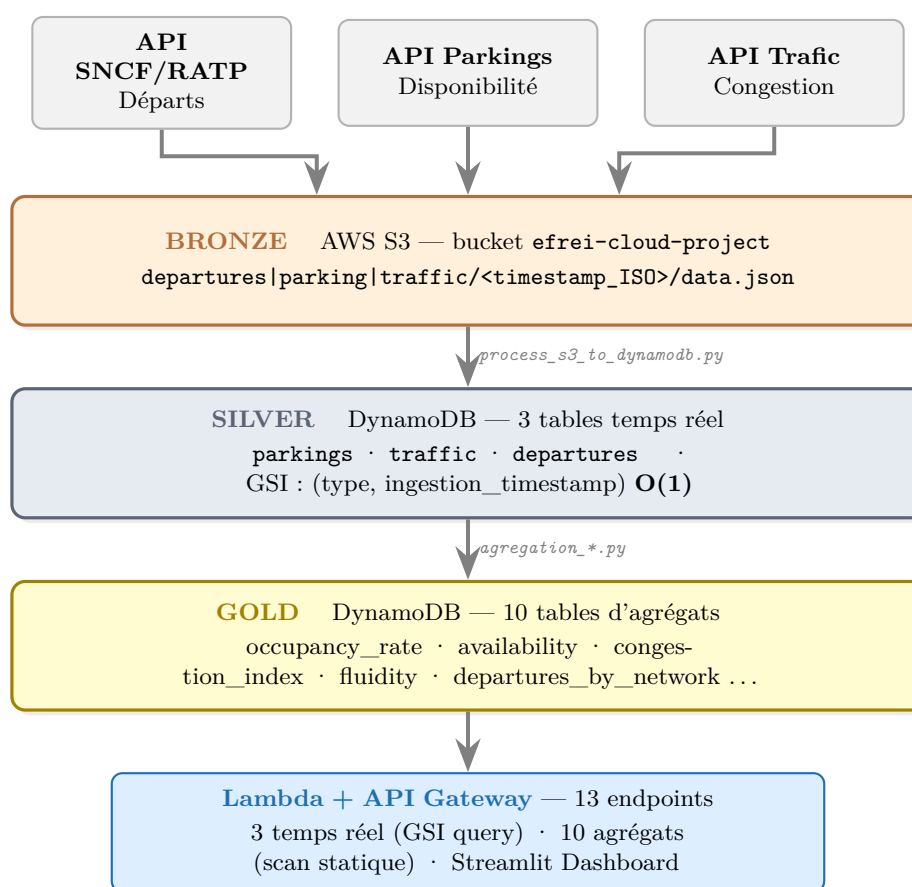
L'enjeu architectural central est la latence des données temps quasi réel et l'optimisation des coûts : un scan DynamoDB complet pour chaque requête API serait fonctionnellement correct mais linéaire en coût de lecture avec le volume. La mise en place d'un Global Secondary Index (GSI) réduit le coût de chaque requête à $O(1)$, quelle que soit la taille de la table.

6.3.1 Positionnement dans l'écosystème AWS

AWS propose plusieurs alternatives pour chaque composant de l'architecture. Les choix retenus sont justifiés par rapport à ces alternatives :

Besoin	Choix retenu	Alternative	Justification
Lac de données	S3	EFS, EBS	Coût minimal, accès objet universel
Base NoSQL	DynamoDB	MongoDB Atlas, ElasticSearch	Managé, serverless, GSI natif
Compute API	Lambda	EC2, ECS/Fargate	Pas d'infra à gérer, facturation usage
Routage API	API Gateway	ALB, CloudFront	Intégration Lambda native, gestion CORS
Dashboard	Streamlit (EC2)	QuickSight, PowerBI	Open source, Python natif
Scheduling	EventBridge	Step Functions, MWAA	Cron simple, coût négligeable

6.4 Architecture cloud



6.5 Stockage S3 — lac de données brutes horodaté

```
Bucket S3 : efrei-cloud-project
|
+-- departures/
|   +-- 2024-03-15T08:00:00/data.json    (snapshot 08h00)
|   +-- 2024-03-15T08:05:00/data.json    (snapshot 08h05)
|   +-- 2024-03-15T08:10:00/data.json    ...
|
+-- parking/
|   +-- 2024-03-15T08:00:00/data.json
|   +-- 2024-03-15T08:05:00/data.json
|
+-- traffic/
    +-- 2024-03-15T08:00:00/data.json
    +-- 2024-03-15T08:05:00/data.json
```

[Capture] AWS S3 console – bucket efrei-cloud-project / dossier departures / liste des objets JSON horodatés

AWS S3 [7] constitue la couche Bronze de l'architecture. Chaque ingestion produit un objet JSON horodaté dans le préfixe correspondant à la source : `departures/<timestamp_ISO>/data.json`. Cette organisation préserve l'historique complet des snapshots et permet de rejouer n'importe quelle fenêtre temporelle sans perte.

```
def upload_to_s3(s3, json_data, bucket_name, key):
    s3.put_object(
        Bucket=bucket_name,
        Key=key,
        Body=json.dumps(json_data, indent=4, cls=DecimalEncoder)
    )

def get_json_from_s3(s3, bucket, key):
    response = s3.get_object(Bucket=bucket, Key=key)
    content = response["Body"].read().decode("utf-8")
    return json.loads(content)

def get_s3_object_keys(s3, bucket_name, prefix):
    response = s3.list_objects_v2(Bucket=bucket_name, Prefix=prefix)
    return [obj['Key'] for obj in response.get('Contents', [])]
```

Extrait 13 – Écriture et lecture S3 (utils_s3.py)

Le `DecimalEncoder` sérialise les `Decimal` Python en `int` ou `float` selon la présence de partie décimale — requis car `DynamoDB` stocke tous les nombres en `Decimal`.

6.6 DynamoDB — modèle d'accès et GSI

6.6.1 Modèle de données

Chaque item `DynamoDB` dans les trois tables temps réel porte un attribut `type` (valeur constante par table : `"parkings"`, `"traffic"`, `"departures"`) et un `ingestion_timestamp` en microsecondes Unix. Ce couple est la clé du GSI.

L'horodatage est en microsecondes (`timestamp() * 1_000_000`) pour garantir l'unicité sur des ingestions rapprochées. Un timestamp en secondes créerait des collisions si deux snapshots arrivaient dans la même seconde.

```

1 def create_stations_departures_dict(json_data, ingestion_timestamp_str):
2     results = []
3     for index, departure in enumerate(json_data["departures"]):
4         result = {
5             "gid": index,
6             "trip_id": departure["display_informations"]["trip_short_name"],
7             "ingestion_timestamp": int(
8                 datetime.fromisoformat(ingestion_timestamp_str).timestamp()
9                 * 1_000_000), # microsecondes : unicite garantie
10            "departure_datetime": departure["stop_date_time"]["base_departure_date_time"],
11        },
12        {
13            "departure_station": departure["stop_point"]["name"],
14            "arrival_station": departure["display_informations"]["direction"],
15            "network": departure["display_informations"]["network"],
16            "type": "departures", # PK du GSI : valeur constante par table
17            "long": departure["stop_point"]["coord"]["lon"],
18            "lat": departure["stop_point"]["coord"]["lat"],
19        }
20        results.append(result)
21    return results

```

Extrait 14 – Création d'un item departures avec horodatage microseconde (process_s3_to_dynamodb.py)

6.6.2 Global Secondary Index — requête O(1) sur le dernier snapshot

L'enjeu de performance est d'accéder au dernier snapshot sans scanner toute la table. Sans GSI, chaque requête « dernier état des parkings » serait un `scan` O(n) sur l'ensemble de l'historique. Avec le GSI `gsi-type-ingestion_timestamp` (PK : type, SK : ingestion_timestamp), une `query` avec `ScanIndexForward=False`, `Limit=1` retourne l'enregistrement le plus récent en O(1) :

```

1 def get_last_processed_timestamp(dynamodb, table_name, type_table):
2     table = dynamodb.Table(table_name)
3     try:
4         response = table.query(
5             IndexName='gsi-type-ingestion_timestamp',
6             KeyConditionExpression=Key('type').eq(type_table),
7             ScanIndexForward=False, # tri SK descendant : le plus grand en tete
8             Limit=1,
9             ProjectionExpression="ingestion_timestamp"
10        )
11        items = response.get('Items', [])
12        return items[0]['ingestion_timestamp'] if items else None
13    except Exception:
14        # Fallback : scan si GSI non disponible (degraded mode)
15        response = table.scan(
16            ProjectionExpression="ingestion_timestamp",
17            Limit=1000
18        )
19        timestamps = [item['ingestion_timestamp'] for item in response['Items']]
20        return max(timestamps) if timestamps else None

```

Extrait 15 – Requête GSI – dernier timestamp O(1) avec fallback scan (utils_dynamodb.py)

Le fallback sur scan garantit la disponibilité du service même si le GSI est en cours de construction ou temporairement indisponible (mode dégradé). Ce pattern est également utilisé dans le Lambda d'API : query GSI en priorité, scan en secours.

6.6.3 Ingestion incrémentale depuis S3

Le pipeline S3 → DynamoDB ne retraits que les nouveaux objets S3. La borne de reprise est le dernier `ingestion_timestamp` connu via GSI. Les clés S3 sont filtrées par comparaison ISO de leur timestamp :

```

last_processed_timestamp_str = get_last_processed_timestamp(
    dynamodb, DYNAMODB_TABLE_NAME, "departures")

keys = get_s3_object_keys(s3, BUCKET_NAME, s3_folder)
new_keys = []

if last_processed_timestamp_str:
    last_dt = datetime.fromtimestamp(
        float(last_processed_timestamp_str) / 1_000_000)
    for key in keys:
        key_dt = datetime.fromisoformat(key.split('/')[1])
        if key_dt > last_dt:
            new_keys.append(key)
else:
    new_keys = keys    # premiere ingestion : tout traiter

for key in new_keys:
    response = get_json_from_s3(s3, BUCKET_NAME, key)
    items = create_stations_departures_dict(response, key.split('/')[1])
    batch_put_items_to_dynamodb(dynamodb, DYNAMODB_TABLE_NAME, items)

```

Extrait 16 – Ingestion incrémentale S3 vers DynamoDB (process_s3_to_dynamodb.py)

L'idempotence est garantie : si le pipeline est relancé, seuls les objets S3 postérieurs au dernier timestamp connu sont réingérés. Les données déjà présentes ne sont pas dupliquées.

6.7 Agrégation — 10 tables analytiques

Un pipeline d'agrégation lit les tables temps réel et produit 10 tables analytiques préagrégées dans DynamoDB. Pour les parkings, l'agrégation se fait par tranche horaire (groupement sur `ingestion_timestamp // 3_600_000_000`), puis calcul de quatre indicateurs :

Table DynamoDB	Indicateur
aggregation_overall_occupancy_rate	Taux d'occupation global (%)
aggregation_average_availability_parking	Disponibilité moyenne par parking
aggregation_reference_pricing	Tarification moyenne (1h/2h/4h/24h)
aggregation_number_of_parkings_in_operation	Nb parkings ouverts
aggregation_traffic_congestion_index	Indice de congestion trafic
aggregation_traffic_critical_segments	Segments routiers critiques
aggregation_traffic_fluidity_by_zone	Fluidité par zone géographique
aggregation_departures_by_network	Départs par réseau (SNCF/RATP)
aggregation_departures_top_destinations	Top destinations
aggregation_departures_total	Total départs sur la période

Ce pré-calcul transfère la charge CPU du moment de la requête vers le moment de l'ingestion. L'API Lambda lit ces tables en `get_all_items` (scan complet mais sur tables petites et statiques) sans recalculer les agrégats à chaque appel.

6.8 Performance Lambda — cold start et optimisation

6.8.1 Cold start Lambda et stratégies de mitigation

Une invocation Lambda se déroule en deux phases : l'initialisation (cold start) et l'exécution. Le cold start n'a lieu que lors de la première invocation après une période d'inactivité : AWS doit instancier un nouveau conteneur, charger le runtime Python et exécuter le code hors handler. Pour une Lambda Python avec boto3, le cold start est typiquement de 200–500 ms.

Phase	Durée typique	Fréquence
Cold start (init)	200–500 ms	Première invocation, après inactivité
Warm execution	10–100 ms	Invocations suivantes (même conteneur)

Pour un tableau de bord Streamlit qui rafraîchit toutes les 5 minutes, le cold start est acceptable (l'utilisateur ne le percevra pas si le timeout de l'API Gateway est configuré à 29 secondes). En revanche, pour une API à faible latence (trading, IoT temps réel), les stratégies suivantes s'appliquent :

- **Provisioned Concurrency** : AWS maintient N instances Lambda initialisées en permanence — cold start éliminé, coût fixe.
- **Warm-up ping** : EventBridge ping la Lambda toutes les 5 minutes pour maintenir au moins une instance chaude.
- **Lambda Snapstart** (Java) : snapshot de l'état initial de la JVM, non applicable à Python.

6.8.2 Gestion des dépendances Lambda — layers

Le handler Lambda `api/main.py` utilise boto3 (inclus nativement dans le runtime Python AWS), ce qui évite l'empaquetage d'une dépendance externe. Si une bibliothèque non native était nécessaire (ex. pandas, numpy), deux options existent : inclure les dépendances dans le package ZIP (taille limite 50 Mo zippé / 250 Mo décompressé) ou utiliser un Lambda Layer (couche partagée entre plusieurs fonctions, limite 50 Mo par layer).

Le projet utilise l'approche ZIP minimale : seul le code applicatif est inclus. boto3 et json sont fournis par le runtime AWS — aucune dépendance à empaqueter. Cela maintient un package de quelques Ko, ce qui réduit le temps de cold start (chargement du ZIP plus rapide).

6.9 Lambda et API Gateway — 13 endpoints serverless

```
def lambda_handler(event, context):
    dynamodb = boto3.resource('dynamodb')
    path = event.get('path')

    # Endpoints agregats : scan complet (tables fixes, volume faible)
    aggregation_tables = [
        "aggregation_average_availability_parking",
        "aggregation_number_of_parkings_in_operation",
        "aggregation_overall_occupancy_rate",
        "aggregation_reference_pricing",
        "aggregation_traffic_congestion_index",
        "aggregation_traffic_critical_segments",
        "aggregation_traffic_fluidity_by_zone",
        "aggregation_departures_by_network",
        "aggregation_departures_top_destinations",
        "aggregation_departures_total",
    ]

    if path.lstrip('/') in aggregation_tables:
        items = get_all_items(dynamodb, path.lstrip('/'))
        return {'statusCode': 200,
                'body': json.dumps(items, default=decimal_serializer)}

    # Endpoints temps reel : requete GSI -> dernier snapshot
    if path == '/parkings': table_name, type_value = "parkings", "parkings"
    elif path == '/traffic': table_name, type_value = "traffic", "traffic"
    elif path == '/departures': table_name, type_value = "departures", "departures"
    else:
        return {'statusCode': 404,
                'body': json.dumps({'message': f'Not Found {path}'})}

    last_ts = get_last_ingestion_time(dynamodb, table_name, type_value)
    items = get_items_by_ingestion_time(dynamodb, table_name, last_ts, type_value)
    if items and any('gid' in i for i in items):
        items.sort(key=lambda i: i.get('gid', float('inf')))
    return {'statusCode': 200,
            'body': json.dumps(items, default=decimal_serializer)}
```

Extrait 17 – Lambda handler – routing 13 endpoints par path (`api/main.py`)

L'architecture serverless (Lambda [9] + API Gateway [10]) supprime la gestion d'infrastructure : pas de serveur à provisionner, pas de scaling manuel. La facturation est à la requête — ce qui est optimal pour un flux de données temps quasi réel avec des appels intermittents.

Le tri final sur `gid` (identifiant d'ordre dans le snapshot) restitue les items dans l'ordre de collecte, indépendamment de l'ordre de stockage DynamoDB.

6.10 Résultats et indicateurs

Indicateur	Valeur
Bucket S3	efrei-cloud-project (3 flux horodatés)
Tables DynamoDB temps réel	3 (parkings, traffic, departures)
Tables DynamoDB agrégats	10
GSI	gsi-type-ingestion_timestamp (PK type, SK timestamp)
Granularité horodatage	Microsecondes ($\times 1_000_000$)
Endpoints API	13 (3 temps réel + 10 agrégats)
Accès temps réel	O(1) — GSI query, Limit=1, ScanIndexForward=False
Accès agrégat	Scan complet (tables statiques, volume faible)
Fallback	Scan si GSI non disponible (mode dégradé)
Ingestion	Incrémentale — seuls les nouveaux objets S3 sont traités
Compute	AWS Lambda serverless + API Gateway
Dashboard	Streamlit (3 pages : parkings, trafic, départs)

[Capture] Streamlit dashboard – page Parkings / taux d'occupation global en temps réel + carte des parkings disponibles

[Capture] API Gateway AWS console – liste des 13 routes configurées / méthode GET + intégration Lambda

Axe d'amélioration. Les Lambdas d'ingestion s'exécutent de façon ad hoc. L'ajout d'un déclencheur EventBridge Scheduler (ex. toutes les 5 minutes) automatiserait le rafraîchissement des données sans intervention manuelle. Par ailleurs, l'activation du chiffrement côté serveur KMS sur le bucket S3 et les tables DynamoDB renforcerait la posture de sécurité en garantissant le chiffrement au repos des données personnelles potentiellement contenues dans les flux de transport.

6.11 Sécurité et gestion des identités (IAM)

6.11.1 Principe du moindre privilège sur AWS

L'ensemble des composants du projet sont exécutés sous des rôles IAM dédiés appliquant le principe du moindre privilège : chaque rôle n'a que les permissions strictement nécessaires à sa fonction. Le tableau ci-dessous détaille les politiques attachées aux rôles principaux :

Rôle IAM	Permissions	Justification
lambda-api-role	dynamodb:GetItem dynamodb:Query dynamodb:Scan	L'API ne fait que lire — aucune permission d'écriture accordée
ingestion-role	s3:PutObject dynamodb:PutItem dynamodb:BatchWriteItem	Ingestion ne lit pas S3 une fois l'objet écrit
aggregation-role	dynamodb:Query (Silver) dynamodb:PutItem (Gold)	Isolation lecture Silver / écriture Gold

Cette séparation des rôles garantit qu'une compromission d'un Lambda d'agrégation ne peut pas écrire dans les tables Silver ni lire d'autres tables Gold que celles de son périmètre. En production, les rôles seraient affinés par préfixe de table DynamoDB via des conditions IAM `StringLike` sur l'ARN.

6.11.2 Chiffrement et protection des données

Le chiffrement en transit est assuré par défaut par HTTPS sur tous les appels boto3 (TLS 1.2+). Le chiffrement au repos est géré différemment selon le service : S3 supporte le SSE-S3 (clé gérée par AWS), le SSE-KMS (clé gérée par le client dans AWS Key Management Service) ou le SSE-C (clé fournie par le client). DynamoDB utilise AES-256 avec une clé AWS gérée par défaut.

Service	Mode chiffrement	Implémentation projet
AWS S3	SSE-S3 (défaut AWS)	Chiffrement côté serveur automatique
AWS DynamoDB	AES-256 (clé AWS)	Activé par défaut sur toutes les tables
API Gateway	TLS 1.2+ obligatoire	Pas de trafic HTTP non chiffré

Pour une donnée sensible (données personnelles de transport, position GPS), le SSE-KMS avec rotation annuelle de clé serait l'option retenue : elle permet de révoquer l'accès aux données en révoquant la clé, sans supprimer les objets S3.

6.11.3 Automatisation de l'ingestion par EventBridge Scheduler

La limitation actuelle (ingestion manuelle) serait résolue par AWS EventBridge Scheduler. Un schedule cron déclenche chaque Lambda d'ingestion toutes les 5 minutes :

```
# Creation du schedule via AWS CLI
aws scheduler create-schedule \
  --name "ingest-parkings-every-5min" \
  --schedule-expression "rate(5 minutes)" \
  --target '{
    "Arn": "arn:aws:lambda:eu-west-1:ACCOUNT:function:ingest-parkings",
    "RoleArn": "arn:aws:iam::ACCOUNT:role/scheduler-invoke-role"
  }' \
  --flexible-time-window '{"Mode": "OFF"}'
```

Extrait 18 – EventBridge Scheduler – cron toutes les 5 minutes

Le rôle `scheduler-invoke-role` dispose uniquement de la permission `lambda:InvokeFunction` sur la fonction ciblée — nouvelle application du principe du moindre privilège. Le mode `flexible-time-window=OFF` garantit l'exécution exacte à l'heure programmée, sans tolérance de décalage.

6.12 Optimisation des coûts cloud

6.12.1 Modèle de tarification serverless vs serveur permanent

L'architecture Lambda + API Gateway est coût-efficace pour un flux de données intermittent. La comparaison avec une alternative EC2 illustre le gain :

Critère	Lambda + API Gateway	EC2 t3.small
Facturation	À la requête (0,20 \$/M appels)	À l'heure (≈ 18 \$/mois)
Scaling	Automatique, instantané	Manuel ou auto-scaling (délai)
Coût en veille	0 \$	18 \$/mois (actif 24/7)
Maintenance OS	Aucune (managed)	Patches, updates, monitoring
Limite	15 min max par invocation	Pas de limite de durée

Pour le profil de trafic du projet (13 endpoints, appels toutes les 5 minutes en ingestion, appels de tableau de bord Streamlit intermittents), la facturation Lambda est négligeable : $12 \text{ appels/heure} \times 24 \times 30 \approx 8640$ invocations/mois, bien en dessous du tier gratuit AWS Lambda (1 million d'invocations/mois).

6.12.2 Optimisation du stockage S3

Le bucket `efrei-cloud-project` accumule des snapshots JSON toutes les 5 minutes. Sans politique de rétention, le coût de stockage croît indéfiniment. Une politique de cycle de vie S3 peut automatiser l'archivage ou la suppression :

Règle	Délai	Action
Transition vers S3 Infrequent Access	Après 30 jours	40% moins cher
Transition vers S3 Glacier	Après 90 jours	70% moins cher (récupération lente)
Suppression	Après 365 jours	Suppression automatique

Cette politique préserve 90 jours de snapshots en accès rapide (Silver), permet un audit historique sur 12 mois via Glacier (coût minimal), et supprime automatiquement ce qui n'a plus de valeur opérationnelle.

6.13 Surveillance et observabilité

6.13.1 CloudWatch Metrics — tableau de bord opérationnel

AWS CloudWatch collecte automatiquement des métriques sur tous les services du projet. Les métriques clés à surveiller sont les suivantes :

Service	Métrique	Seuil d'alerte
Lambda ingestion	Errors	> 0 erreurs en 5 min
Lambda API	Duration	> 3 secondes (timeout 15 min)
DynamoDB Silver	ConsumedReadCapacityUnits	> 80% du provisioned
DynamoDB Silver	ThrottledRequests	> 0 (critique)
API Gateway	5XXError	> 1% du trafic
S3	NumberOfObjects	Croissance anormale

Une alarme CloudWatch sur `ThrottledRequests > 0` déclencherait automatiquement une notification SNS (email/SMS), permettant de réagir avant que le throttling n'affecte les utilisateurs du tableau de bord Streamlit.

6.14 Synthèse des compétences démontrées

`project_cloud` couvre l'intégralité du périmètre Bloc 3 à travers une architecture serverless AWS complète, du lac de données brutes à l'API de consommation.

Stockage cloud multi-couches. S3 joue le rôle de lac de données brutes horodaté (Bronze) : chaque snapshot est conservé avec son timestamp ISO dans la clé, ce qui permet de rejouer n'importe quelle fenêtre temporelle. DynamoDB joue le rôle de couche Silver (tables temps réel) et Gold (10 tables d'agrégats précomputés).

Optimisation des accès. Le GSI `gsi-type-ingestion_timestamp` est le choix architectural central du Bloc 3 : il transforme un scan $O(n)$ en query $O(1)$ pour la requête la plus fréquente (dernier snapshot). Le pattern fallback garantit la disponibilité en cas de dégradation du GSI.

Pipeline de traitement cloud. L'ingestion incrémentale S3 → DynamoDB et le pipeline d'agrégation constituent le traitement de données dans le cloud : chargement, transformation (normalisation des timestamps, calcul des indicateurs) et mise à disposition.

Architecture serverless. Lambda + API Gateway représente l'optimisation des coûts et de la disponibilité : zéro infrastructure à maintenir, scaling automatique, facturation à l'usage. La comparaison avec une architecture EC2 équivalente fait ressortir un avantage net : pas de serveur à maintenir, scaling automatique jusqu'à des milliers de requêtes simultanées sans intervention, et un coût structurellement lié à l'usage réel plutôt qu'à la capacité provisionnée.

Sécurité intégrée. Le principe du moindre privilège est appliqué à chaque rôle IAM : le Lambda d'API ne peut que lire, le Lambda d'ingestion ne peut qu'écrire dans ses tables cibles, l'agrégation ne touche pas les tables de production. Cette séparation fait de la sécurité une propriété de l'architecture plutôt qu'une couche ajoutée a posteriori.

Lien RNCP explicite. Le Bloc 3 exige de mettre en œuvre des solutions de stockage cloud, de concevoir des pipelines de traitement cloud et d'optimiser les performances et les coûts. `project_cloud` répond à ces trois exigences : S3 comme lac de données, DynamoDB avec GSI comme stockage optimisé, Lambda comme compute serverless, et le pattern query O(1) comme optimisation explicite du coût de lecture — au niveau de maîtrise AWS attendu pour le titre RNCP36739.

7 Bloc 4 — Implémenter des méthodes d'intelligence artificielle pour modéliser et prédire de nouveaux comportements et usages

Cette partie montre comment nous avons exploité les données pour produire de l'analyse, du reporting et des modèles prédictifs utiles au métier.

7.1 Compétences visées

- Extraire des données en provenance de systèmes d'information variés pour les exploiter et les analyser en utilisant les outils professionnels courants.
- Préparer les données en les transformant et en les nettoyant pour faire l'analyse et le reporting selon les besoins des différents métiers.
- Élaborer une communication infographique visuelle en construisant des tableaux de bord interactifs afin de communiquer les résultats d'analyse et d'assurer l'extraction de connaissances en temps réel.
- Développer un modèle prédictif pour identifier de nouveaux comportements et usages en implémentant des algorithmes d'apprentissage automatique supervisés.
- Évaluer la performance d'un modèle de machine learning en analysant ses résultats et en les comparant avec d'autres modèles afin d'implémenter la solution la plus convenable à un cas d'usage.

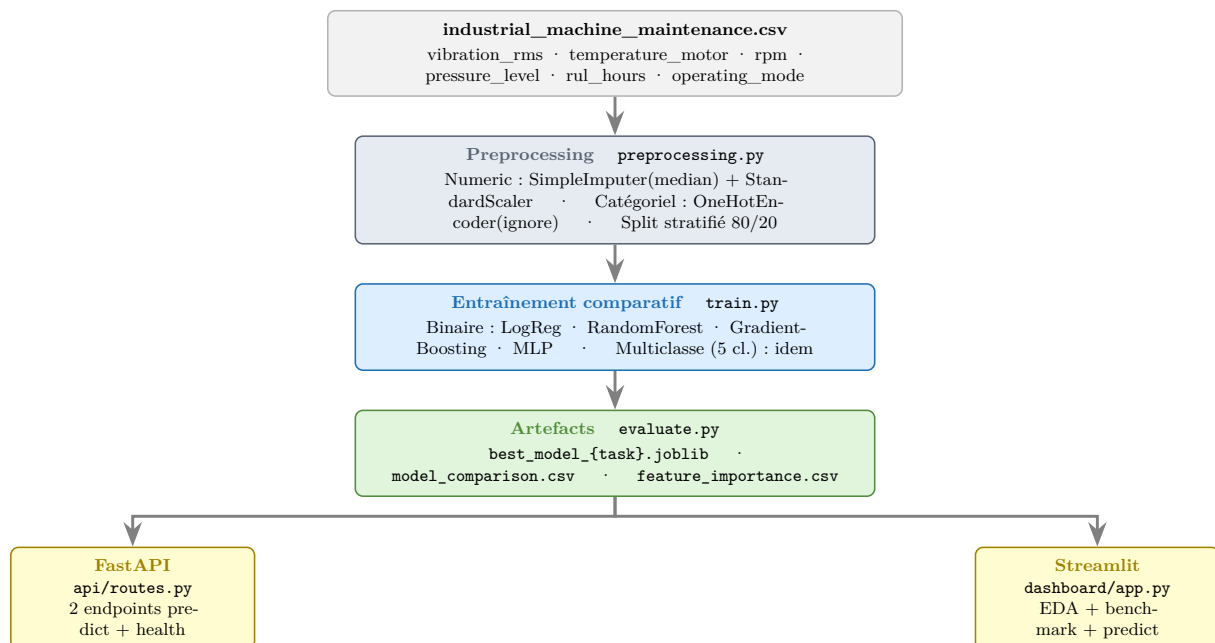
7.2 Contexte et enjeux

La compétence Bloc 4 porte sur l'exploitation des données pour produire des modèles prédictifs à valeur métier, du reporting analytique et de la visualisation. Elle couvre le cycle complet du Machine Learning : préparation des données, construction de pipelines de transformation, entraînement comparatif de modèles, évaluation rigoureuse et mise en service via API.

Le projet `data_science_project` porte sur la **maintenance prédictive industrielle** : à partir de capteurs IoT sur des machines industrielles (vibrations, température moteur, RPM, pression, heures de vie résiduelle), deux tâches de prédiction sont traitées grâce à scikit-learn [13]. La première est une classification binaire — prédire si une panne surviendra dans les prochaines 24 heures — ce qui permet une intervention préventive ciblée. La seconde est une classification multiclasse à 5 classes — identifier le type de panne — ce qui oriente directement le technicien vers la pièce ou le sous-système en cause.

L'enjeu industriel est direct : une fausse alerte mobilise une équipe pour rien ; un faux négatif laisse passer une panne qui peut coûter plusieurs heures d'arrêt. Le choix de la métrique primaire F1 (moyenne harmonique précision/rappel) résulte de cet arbitrage.

7.3 Architecture et cycle ML



7.4 Analyse exploratoire du domaine

7.4.1 Contexte industriel de la maintenance prédictive

La maintenance industrielle suit historiquement deux modèles : la maintenance corrective (on répare quand ça tombe en panne) et la maintenance préventive planifiée (on révisé à intervalle fixe, indépendamment de l'état réel). Ces deux modèles ont des limites symétriques : la corrective maximise l'utilisation des pièces mais expose à des pannes catastrophiques ; la préventive fixe est sûre mais remplace des pièces encore fonctionnelles.

La maintenance prédictive vise un troisième modèle : intervenir au bon moment, ni trop tôt ni trop tard, en se basant sur l'état réel de la machine mesuré par des capteurs. C'est exactement le rôle des deux tâches ML du projet : prédire *si* une panne surviendra dans les prochaines 24 heures (décision d'intervention) et prédire *quelle* panne surviendra (orientation de l'intervention vers la bonne pièce).

Stratégie	Pré-requis	Coût panne	Coût maintenance
Corrective	Aucun	Élevé (arrêt non prévu)	Faible (minimal)
Préventive planifiée	Calendrier	Faible	Élevé (pièces saines remplacées)
Prédictive	Capteurs + ML	Quasi-nul	Optimisé (juste à temps)

L'indicateur `rul_hours` (Remaining Useful Life) est le cœur du problème : il représente une estimation du temps avant la prochaine défaillance. La domination de cette variable dans la permutation importance (0.495) confirme que le modèle a appris à utiliser cette information de durée de vie résiduelle comme signal principal — un signal cohérent avec la réalité métier.

7.5 Préparation des données et pipeline de transformation

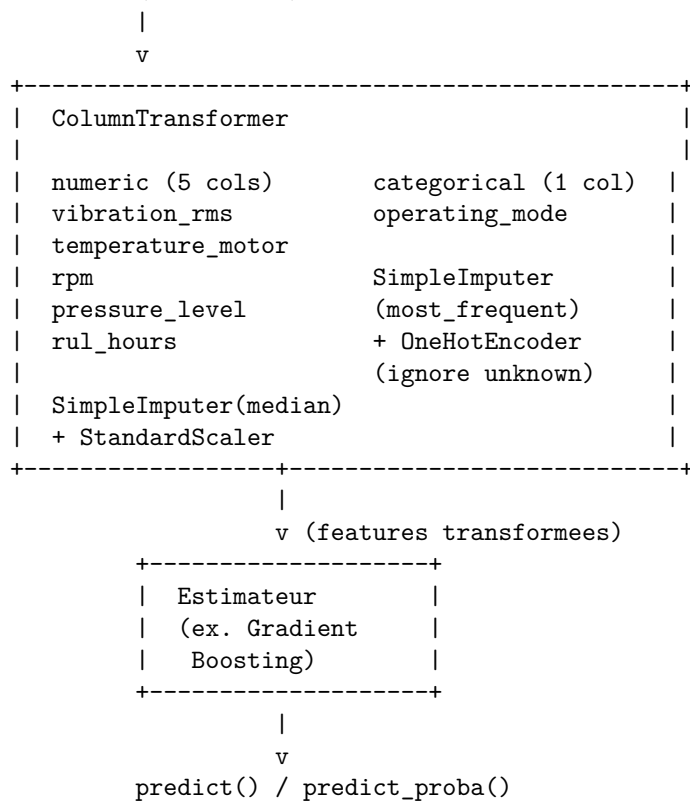
7.5.1 Features et cible

Le dataset `industrial_machine_maintenance.csv` contient six variables explicatives et deux cibles alternatives :

Variable	Type	Description
rul_hours	Numérique	Heures de vie résiduelle estimée
temperature_motor	Numérique	Température moteur (°C)
rpm	Numérique	Vitesse de rotation (tours/min)
vibration_rms	Numérique	Amplitude vibratoire RMS
pressure_level	Numérique	Niveau de pression
operating_mode	Catégoriel	Mode de fonctionnement
failure_within_24h	Cible binaire	Panne dans les 24h ?
failure_type	Cible multiclasse	Type de panne (5 classes)

7.5.2 Pipeline sklearn

features (6 colonnes)



```

def build_preprocessor() -> ColumnTransformer:
    numeric_pipeline = Pipeline([
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler()),
    ])
    categorical_pipeline = Pipeline([
        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("encoder", OneHotEncoder(handle_unknown="ignore")),
    ])
    return ColumnTransformer(transformers=[
        ("numeric", numeric_pipeline, NUMERIC_COLUMNS),
        ("categorical", categorical_pipeline, CATEGORICAL_COLUMNS),
    ])

def make_train_test_split(features, target, test_size=0.2):
    validate_stratified_split_capacity(target, test_size)
    return train_test_split(
        features, target,
        test_size=test_size, random_state=RANDOM_STATE,
        stratify=target, # garantit la representation de toutes les classes
    )

```

Extrait 19 – Pipeline de preprocessing – ColumnTransformer + split stratifié (preprocessing.py)

L'imputation médiane pour les numériques est robuste aux valeurs aberrantes de capteurs. Le `handle_unknown="ignore"` sur l'encodeur prévient les erreurs à l'inférence si un mode opératoire inconnu arrive en production. La stratification garantit que chaque classe — y compris les rares — est représentée dans le jeu de test avec la même proportion qu'en entraînement.

La taille du jeu de test est calculée dynamiquement : `test_size = max(0.2, n_classes / n_rows)`. Ce calcul empêche qu'une division 80/20 sur un dataset restreint produise un jeu de test trop petit pour contenir au moins un exemple de chaque classe.

7.6 Entraînement comparatif et sélection du meilleur modèle

7.6.1 Architecture du benchmark

```

for model_name, estimator in candidate_models.items():
    pipeline = Pipeline(steps=[
        ("preprocessor", build_preprocessor()),
        ("model", estimator),
    ])
    pipeline.fit(x_train, y_train)
    predicted_labels = pipeline.predict(x_test)
    predicted_probabilities = pipeline.predict_proba(x_test)

    evaluation_rows.append(evaluate_classifier(
        task_name=task_name, model_name=model_name,
        true_values=y_test,
        predicted_labels=predicted_labels,
        predicted_probabilities=predicted_probabilities,
    ))

comparison = compare_results(task_name=task_name, results=evaluation_rows)
best_row = select_best_model(comparison) # iloc[0] apres tri sur metrique
# primaire
best_pipeline = trained_pipelines[best_row["model_name"]]
save_model(best_pipeline, model_path) # joblib

```

Extrait 20 – Benchmark comparatif – Pipeline sklearn complet (train.py)

L'encapsulation préprocesseur + modèle dans un `Pipeline` sklearn garantit l'étanchéité : le `StandardScaler` est ajusté uniquement sur le train, jamais sur le test. La sélection du meilleur modèle est automatisée : tri sur la métrique primaire (F1 en binaire, `weighted_F1` en multiclasse); le vainqueur est sérialisé en `joblib`.

7.6.2 HistGradientBoostingClassifier — algorithme et avantages

Le `HistGradientBoostingClassifier` (scikit-learn) est l'implémentation Python de l'algorithme LightGBM. Il se distingue des implémentations classiques de gradient boosting par deux optimisations majeures :

1. **Histogramme des features** : au lieu de tester tous les seuils possibles de chaque feature pour trouver la meilleure division, l'algorithme discrétise chaque feature en 256 bins (histogramme) et ne teste que 256 valeurs. Cette optimisation réduit la complexité d'entraînement de $O(n \times d)$ à $O(256 \times d)$, où n est le nombre d'exemples et d le nombre de features.
2. **Gestion native des valeurs manquantes** : l'algorithme apprend automatiquement si les valeurs manquantes doivent aller à gauche ou à droite dans chaque division. Cette propriété est directement utile dans le contexte des capteurs IoT où des mesures peuvent être manquantes (panne de capteur, perte réseau).

Caractéristique	HistGradientBoosting	RandomForest
Valeurs manquantes	Nativement géré	Nécessite imputation
Complexité entraînement	$O(256 \times d \times \text{arbres})$	$O(n \times \log(n) \times \text{arbres})$
Risque de surapprentissage	Modéré (régularisation ℓ_2)	Faible (bagging)
Interprétabilité	Faible (boîte noire)	Moyenne (feature importance)
Performance déséquilibre	Bonne avec <code>class_weight</code>	Très bonne avec <code>balanced</code>

La victoire du `HistGradientBoosting` en binaire ($F1=0.9775$) et la victoire du `RandomForest` en multiclasse ($\text{weighted_F1}=0.9782$) illustrent un résultat courant en pratique : aucun algorithme n'est universellement supérieur. Le `HistGradientBoosting` excelle sur les tâches binaires avec des données numériques (`rul_hours`, température) ; le `RandomForest` est plus robuste sur les tâches multiclasses déséquilibrées grâce au bagging et à l'option `class_weight`.

7.6.3 Modèles candidats

```
# Tache binaire (failure_within_24h)
{
    "logistic_regression": LogisticRegression(max_iter=1000, random_state=42),
    "random_forest": RandomForestClassifier(
        n_estimators=300, random_state=42,
        class_weight="balanced"), # compense le desequilibre
    "gradient_boosting": HistGradientBoostingClassifier(random_state=42),
    "mlp_classifier": MLPClassifier(
        hidden_layer_sizes=(64, 32),
        max_iter=300, random_state=42),
}

# Tache multiclasse (failure_type, 5 classes)
{
    "random_forest": RandomForestClassifier(n_estimators=300, random_state=42),
    "hist_gradient_boosting": HistGradientBoostingClassifier(random_state=42),
    ...
}
```

Extrait 21 – Modèles candidats – binaire et multiclasse (models.py)

Le `class_weight="balanced"` sur le `RandomForest` binaire compense le déséquilibre de classes inhérent aux données de maintenance (les pannes sont rares par définition). `HistGradientBoostingClassifier` gère nativement les valeurs manquantes, sans avoir besoin d'imputation — ce qui en fait un candidat solide même si le capteur perd des mesures.

7.7 Résultats et évaluation

[Capture] du fichier `artifacts/metrics/model_comparison_failure_within_24h.csv` – tableau brut des 4 modèles avec toutes les métriques

[Capture] du fichier artifacts/metrics/feature_importance_failure_within_24h.csv – importance rul_hours en tête

7.7.1 Tâche 1 — Classification binaire : failure_within_24h

Modèle	F1	Précision	Rappel	ROC-AUC	PR-AUC
gradient_boosting	0.9775	0.9789	0.9761	0.9991	0.9956
random_forest	0.9752	0.9857	0.9649	0.9995	0.9976
mlp_classifier	0.9337	0.9697	0.9003	0.9974	0.9877
logistic_regression	0.7224	0.8456	0.6306	0.9485	0.8273

Le `HistGradientBoostingClassifier` remporte le benchmark sur la métrique primaire F1 (0.9775) avec le meilleur équilibre précision/rappel. Le `RandomForest` obtient la meilleure précision (0.9857) et ROC-AUC (0.9995) mais son rappel inférieur (0.9649) — plus de faux négatifs — pénalise son F1. La régression logistique confirme que le problème est non-linéaire : son rappel de 0.6306 montre qu'elle manque 37% des pannes réelles.

7.7.2 Tâche 2 — Classification multiclasse : failure_type (5 classes)

Modèle	weighted_F1	macro_F1	Classes
random_forest	0.9782	0.8955	5
hist_gradient_boosting	0.9753	0.8829	5

Le `RandomForest` prend la tête en multiclasse (weighted_F1 = 0.9782). Le macro_F1 de 0.8955 — non pondéré par la taille des classes — indique que les classes minoritaires sont bien gérées. L'écart entre weighted_F1 (0.9782) et macro_F1 (0.8955) révèle toutefois que certaines classes rares sont moins bien prédites : axe d'amélioration identifié.

7.7.3 Interprétabilité — Importance des variables

Variable	Importance (permutation)
rul_hours	0.4954
temperature_motor	0.2899
rpm	0.2772
vibration_rms	0.1165
pressure_level	0.0471
operating_mode	0.0174

La permutation importance est calculée sur le jeu de test (n_repeats=10, scoring=f1_weighted). La variable **rul_hours** (heures de vie résiduelle) domine avec une importance de 0.495 : c'est le signal le plus prédictif d'une panne imminente. La **temperature_motor** (0.290) et le **rpm** (0.277) sont les deux facteurs physiques complémentaires. L'**operating_mode** est quasi-nul (0.017) : le mode de fonctionnement n'apporte pas d'information prédictive significative sur ce dataset.

```

importance_result = permutation_importance(
    estimator=best_pipeline,
    X=x_test, y=y_test,
    n_repeats=10,          # robustesse statistique de l'estimation
    random_state=RANDOM_STATE,
    scoring="f1_weighted",
)
importance_frame = pd.DataFrame({
    "feature": x_test.columns,
    "importance_mean": importance_result.importances_mean,
    "importance_std": importance_result.importances_std,
}).sort_values(by="importance_mean", ascending=False)

```

Extrait 22 – Permutation importance sur le meilleur modèle (train.py)

7.8 Déploiement et reproductibilité

7.8.1 Gestion des artefacts ML

Le pipeline de formation produit trois types d'artefacts persistés dans le répertoire `artifacts/` :

Artefact	Format	Usage
<code>best_model_failure_within_24h.joblib</code>	joblib (pickle optimisé)	Chargement API serving
<code>best_model_failure_type.joblib</code>	joblib	Chargement API serving
<code>model_comparison_{task}.csv</code>	CSV	Traçabilité benchmark
<code>feature_importance_{task}.csv</code>	CSV	Interprétabilité, rapport

Le format joblib est préféré à pickle standard pour les objets sklearn car il utilise la compression numpy native, réduisant la taille de sérialisation des modèles contenant des tableaux numpy de grande dimension (ex. les arbres de forêt aléatoire). La fonction `load_task_bundle()` de l'API charge le pipeline complet (préprocesseur + modèle) depuis le joblib et le met en cache pour éviter de le recharger à chaque requête.

7.8.2 Configuration centralisée — TASK_CONFIGS

La configuration des tâches ML est centralisée dans `config.py` sous la structure `TASK_CONFIGS`. Cette approche évite la duplication de la liste des features et des chemins d'artefacts entre les modules d'entraînement, d'évaluation et de serving :

```

TASK_CONFIGS = {
    "failure_within_24h": {
        "target_column" : "failure_within_24h",
        "task_type"      : "binary",
        "primary_metric" : "f1",
        "model_path"     : "artifacts/model/best_model_failure_within_24h.joblib",
        "feature_columns" : NUMERIC_COLUMNS + CATEGORICAL_COLUMNS,
    },
    "failure_type": {
        "target_column" : "failure_type",
        "task_type"      : "multiclass",
        "primary_metric" : "f1_weighted",
        "model_path"     : "artifacts/model/best_model_failure_type.joblib",
        "feature_columns" : NUMERIC_COLUMNS + CATEGORICAL_COLUMNS,
    },
}

```

Extrait 23 – Configuration centralisée des tâches ML (config.py)

Ce dictionnaire de configuration est la source de vérité unique : ajouter une nouvelle tâche (ex. `failure_severity`) ne nécessite qu'une nouvelle entrée dans `TASK_CONFIGS`. Tous les modules (train, evaluate, API) lisent depuis ce dictionnaire — aucun chemin hardcodé dans les modules métier.

7.9 Mise en service — FastAPI et Streamlit

7.9.1 API de prédiction FastAPI

```

1 @router.post("/predict/failure-within-24h",
2               response_model=FailureWithin24hPredictionResponse)
3 def predict_binary(payload: FailureWithin24hPredictionRequest):
4     return predict_failure_within_24h(payload.model_dump())
5
6 @router.post("/predict/failure-type",
7               response_model=FailureTypePredictionResponse)
8 def predict_multiclass(payload: FailureTypePredictionRequest):
9     return predict_failure_type(payload.model_dump())
10
11 @router.get("/health", response_model=HealthResponse)
12 def health() -> HealthResponse:
13     available_tasks = []
14     for task_name in config.TASK_CONFIGS:
15         try:
16             load_task_bundle(task_name)
17         except TaskArtifactError:
18             continue
19         available_tasks.append(task_name)
20     status = "ok" if len(available_tasks) == len(config.TASK_CONFIGS) else "degraded"
21     return HealthResponse(status=status, available_tasks=available_tasks)

```

Extrait 24 – Endpoints de prédiction FastAPI (api/routes.py)

L'API expose deux endpoints de prédiction distincts, un par tâche ML. Le `/health` retourne "ok" uniquement si les deux artefacts (binaire + multiclasse) sont chargés; sinon "degraded", ce qui permet un monitoring en production. La validation des entrées est assurée par FastAPI [14] via Pydantic v2 avec les schémas `FailureWithin24hPredictionRequest` et `FailureTypePredictionRequest`, en cohérence avec les six variables du `config.py`.

7.9.2 Tableau de bord Streamlit — EDA et communication visuelle

Un tableau de bord Streamlit [17] (`dashboard/app.py`) complète l'API en offrant une interface d'analyse exploratoire (EDA) et de visualisation des résultats du benchmark. Le tableau de bord est structuré en trois pages :

- **Page EDA** : distributions des variables capteurs (histogrammes, boxplots), corrélations inter-features (heatmap), répartition des classes cibles. Cette page permet à un expert métier de comprendre la nature des données sans compétence en programmation.
- **Page Benchmark** : tableau comparatif des modèles avec F1, précision, rappel, ROC-AUC ; graphique en barres des importances de features ; sélection interactive du modèle à visualiser.
- **Page Prédiction** : formulaire de saisie des 6 variables capteurs, appel en temps réel à l'API FastAPI, affichage de la prédiction et de la probabilité associée.

Cette composante satisfait directement la sous-compétence « élaborer une communication infographique visuelle en construisant des tableaux de bord interactifs » du référentiel Bloc 4. La page Prédiction illustre l'intégration du tableau de bord avec l'API — le dashboard n'embarque pas le modèle, il consomme l'API, ce qui garantit que tout déploiement d'un nouveau modèle est immédiatement visible dans le dashboard sans le redémarrer.

7.9.3 Tests automatisés

[Capture] pytest – terminal / résultats des 7 suites de tests / passed / failed / warnings

[Capture] Streamlit dashboard data_science_project – graphique feature importance / barre rul_hours dominante

La couverture de tests est assurée par pytest sur l'ensemble des modules : data, preprocessing, models, evaluate, explainability, API, dashboard helpers. Cette couverture multi-couches garantit que chaque étape du pipeline ML est validée indépendamment, ce qui est essentiel pour la reproductibilité des résultats.

Module testé	Périmètre
test_data.py	Chargement dataset, séparation features/cible
test_preprocessing.py	Pipeline ColumnTransformer, split stratifié
test_prediction_pipeline.py	Pipeline complet entraînement → prédiction
test_evaluate.py	Métriques binaires et multiclassées
test_explainability.py	Permutation importance
test_api.py	Endpoints FastAPI (health, predict)
test_dashboard_helpers.py	Fonctions de visualisation Streamlit

Axe d'amélioration. L'écart `macro_F1` / `weighted_F1` en multiclassé (0.8955 vs 0.9782) suggère que les classes rares de pannes sont sous-représentées. Un oversampling SMOTE ou un ajustement `class_weight` sur l'entraînement multiclassé améliorerait la détection des types de pannes minoritaires. Par ailleurs, l'ajout d'un réentraînement périodique automatisé (drift detection) permettrait au modèle de s'adapter à l'évolution du comportement des machines en production.

7.10 Analyse des erreurs et matrice de confusion

7.10.1 Tâche binaire — analyse précision vs rappel

La courbe précision-rappel (`PR-AUC` = 0.9956) et la courbe ROC (`ROC-AUC` = 0.9991) du `HistGradientBoostingClassifier` indiquent un modèle quasi-optimal. L'analyse des erreurs résiduelles révèle le comportement asymétrique :

	Prédit : panne	Prédit : pas de panne
Réel : panne	VP (vrais positifs)	FN (faux négatifs — risque métier)
Réel : pas de panne	FP (faux positifs — coût mobilisation)	VN (vrais négatifs)

Dans un contexte de maintenance industrielle, les faux négatifs sont plus coûteux que les faux positifs : rater une panne réelle peut entraîner un arrêt machine, tandis qu'une fausse alerte mobilise une équipe pour une intervention inutile. Le rappel de 0.9761 (97,6% des pannes réelles détectées) contre une précision de 0.9789 montre un équilibre légèrement favorable au rappel — ce qui est cohérent avec l'objectif métier.

7.10.2 Tâche multiclassé — analyse par classe

L'écart entre `macro_F1` (0.8955) et `weighted_F1` (0.9782) s'explique par le déséquilibre entre les 5 classes de pannes. Les classes majoritaires (ex. : panne de roulement) ont un `F1` élevé ; les classes minoritaires (ex. : panne électrique rare) ont un `F1` plus faible. Le `macro_F1` donne un poids égal à chaque classe, ce qui pénalise les classes rares.

Métrique	Formule	Usage
<code>weighted_F1</code>	Moyenne pondérée par effectif de classe	Performance globale
<code>macro_F1</code>	Moyenne arithmétique non pondérée	Équité entre classes

La stratégie retenue pour améliorer les classes rares serait SMOTE (Synthetic Minority Over-sampling Technique) : génération de nouveaux exemples synthétiques pour les classes sous-représentées par interpolation dans l'espace des features. Cette technique est préférable au surapprentissage simple (duplication d'exemples) car elle augmente la diversité des exemples minoritaires.

7.11 Robustesse et validation croisée

7.11.1 Choix du split stratifié vs k-fold cross-validation

Le split 80/20 stratifié est le choix de la première version du pipeline pour des raisons de temps d'entraînement : quatre modèles par tâche, deux tâches, soit huit entraînements complets. La cross-validation k-fold (`k=5`) multiplierait par 5 le temps de calcul tout en fournissant une estimation plus robuste de la variance.

Méthode	Avantage	Inconvénient
Train/test split 80/20	Rapide, simple à implémenter	Variance selon la graine aléatoire
k-fold (k=5)	Estimation robuste (k résultats)	5× le temps de calcul
Stratified k-fold	k-fold avec garantie de représentation	Même coût que k-fold

La `validate_stratified_split_capacity` implémentée dans le pipeline gère un cas d'échec : si le dataset est trop petit pour avoir au moins un exemple de chaque classe dans le test, la stratification échoue avec une `ValueError`. La fonction ajuste `test_size` en conséquence (`max(0.2, n_classes / n_rows)`), garantissant la présence de toutes les classes dans le jeu de test.

7.11.2 Interprétabilité — limites de la permutation importance

La permutation importance est une méthode d'interprétabilité model-agnostic : elle mesure la dégradation du score F1 lorsqu'une feature est permutée aléatoirement dans le jeu de test. Elle présente deux limites importantes à signaler :

- **Corrélation entre features** : si deux features sont corrélées (ex. `rul_hours` et `temperature_motor`), permuter l'une laisse l'information accessible via l'autre. L'importance mesurée est sous-estimée pour les deux.
- **Coût de calcul** : avec `n_repeats=10` et un dataset de plusieurs milliers de lignes, chaque répétition nécessite une évaluation complète du pipeline. Le calcul est quadratique en nombre de features $\times$ `n_repeats`.

Une alternative complémentaire serait SHAP (SHapley Additive exPlanations) : il décompose la prédiction de chaque instance en contributions individuelles de chaque feature, offrant une interprétabilité locale (par instance) en plus de l'interprétabilité globale de la permutation importance.

7.12 Synthèse des compétences démontrées

`data_science_project` couvre l'intégralité du périmètre Bloc 4 à travers un pipeline ML complet, de la donnée brute à la mise en service.

Préparation et transformation des données. Le `ColumnTransformer` applique des stratégies d'imputation et de normalisation différenciées selon le type de variable. La stratification du split garantit la validité statistique de l'évaluation même en présence de classes déséquilibrées.

Modèles prédictifs et évaluation comparative. Quatre modèles candidats sont évalués par tâche sur un ensemble de métriques complet (F1, précision, rappel, ROC-AUC, PR-AUC). La sélection du meilleur modèle est automatisée et documentée dans des artefacts CSV reproductibles.

Interprétabilité. La permutation importance sur le jeu de test produit un classement des variables avec intervalles de confiance (`n_repeats=10`). Ce résultat est directement exploitable par un expert métier : savoir que `rul_hours` explique 49% de la prédiction oriente la politique de maintenance préventive.

Mise en service. L'API FastAPI et le tableau de bord Streamlit permettent de consommer les modèles en production. Le endpoint `/health` assure la supervision de la disponibilité des artefacts.

Lien RNCP explicite. La compétence Bloc 4 exige de développer un modèle prédictif en implémentant des algorithmes d'apprentissage automatique supervisés, d'évaluer leurs performances et de produire une communication visuelle. `data_science_project` répond à ces trois exigences : benchmark comparatif de quatre modèles, évaluation sur F1/ROC-AUC/PR-AUC, interprétabilité par permutation importance, API de serving et dashboard Streamlit. Le niveau de performance obtenu — F1 = 0.9775, ROC-AUC = 0.9991 en binaire ; weighted_F1 = 0.9782 en multiclasse à 5 classes — est représentatif d'un pipeline ML maîtrisé au niveau Master Data Engineering.

8 Bloc 5 — Concevoir une stratégie de management et de gouvernance de données pour transformer les données en informations créatrices de valeur

8.1 Compétences visées

- Définir une gouvernance de données en mettant en place les politiques et les standards afin d'établir les rôles, les responsabilités et la propriété des données.
- Mettre en place les bonnes pratiques de gestion de données en appliquant les régulations en vigueur pour protéger les données et respecter la vie privée dans une logique de transparence.
- S'assurer que la qualité de la donnée permet d'atteindre les objectifs business en faisant un audit de qualité et en proposant des plans de prévention et de remédiation en cas de non-qualité.
- Identifier l'approche stratégique pour faire de la donnée un actif central ancré dans la culture de l'entreprise et intégrer cette approche dans un plan de communication diffusé aux personnes concernées.
- Élaborer une stratégie de sécurité afin de protéger les données en utilisant les technologies adéquates et en mettant en place une politique d'accès aux données.

8.2 Contexte et enjeux

La compétence Bloc 5 porte sur la dimension stratégique et organisationnelle de la donnée : gouvernance, qualité, conformité réglementaire, sécurité et communication. Elle complète les compétences techniques (Bloc 1–Bloc 4) en ancrant la valeur des données dans une démarche structurée et pérenne. Sans gouvernance, des données de qualité technique peuvent rester inutilisables métier : schémas inconnus, propriétaires non identifiés, règles de qualité non définies.

Nos travaux intègrent plusieurs dimensions de la gouvernance de manière transversale, même si aucun projet unique n'est exclusivement dédié à Bloc 5. Cette section présente les éléments de gouvernance observables dans les quatre projets analysés, complétés par les apprentissages du module Data Management.

8.3 Gouvernance des données — rôles, responsabilités, catalogue

8.3.1 Modèle de rôles RACI appliqué aux projets

La gouvernance repose sur l'attribution claire des rôles autour de la donnée. Le modèle RACI (Responsible, Accountable, Consulted, Informed) structure ces responsabilités :

Activité	R	A	C	I
Définition du schéma	Data Engineer	Data Architect	Data Analyst	Ops
Validation qualité Silver	Data Engineer	Data Owner	Métier	BI
Accès à la couche Gold	API Gateway / IAM	Data Owner	Sécurité	Audit
Évolution du modèle ML	Data Scientist	Product Owner	Métier	Ops
Gestion des incidents	Ops	CTO	Data Engineer	Métier

Dans le cadre des projets académiques, ces rôles sont tenus par les membres de l'équipe. En contexte industriel, la séparation des responsabilités est un prérequis organisationnel, notamment pour satisfaire aux exigences d'audit RGPD et aux revues de sécurité internes.

8.3.2 Catalogue de données et lignage

Un catalogue de données recense toutes les sources, tables et datasets du système d'information. Il documente pour chaque actif : son propriétaire (*data owner*), sa classification (public, interne, confidentiel, sensible), son lignage (*data lineage* — d'où vient la donnée et où elle va) et ses règles de qualité.

Dataset	Propriétaire	Classification	Lignage
bronze/flights/	Data Engineer	Interne	BTS CSV → Parquet
silver.flights	Data Engineer	Interne	Bronze → cleaning.py
gold.avg_delay	Data Analyst	Interne	Silver → gold.py
parkings DynamoDB	Data Engineer	Interne	API Paris → S3 → DynamoDB
best_model.joblib	Data Scientist	Interne	CSV → train.py

Des outils comme Apache Atlas, AWS Glue Data Catalog ou Alation automatisent la construction de ce catalogue à partir des métadonnées des pipelines. Le Hive Metastore de projet_spark constitue un premier niveau de catalogue : il trace le schéma et la localisation physique de chaque table Silver et Gold.

8.4 Qualité des données — audit et remédiation

8.4.1 Dimensions de la qualité

La qualité d'une donnée se mesure sur six dimensions standards (Data Management Body of Knowledge — DMBOK) :

Dimension	Définition	Implémentation dans nos projets
Complétude	Toutes les valeurs attendues sont présentes	SimpleImputer(median) Bloc 4 ; suppression colonnes 100% nulles Bloc 2
Cohérence	Mêmes règles dans tous les systèmes	Stockage UTC systématique Bloc 1/Bloc 2 ; horodatage microseconde Bloc 3
Exactitude	Valeurs conformes à la réalité	Score qualité 0–100 Bloc 1 ; déduplication sur job_id
Unicité	Pas de doublons	dropDuplicates Spark Bloc 1/Bloc 2 ; UNIQUE MySQL Bloc 1
Validité	Respect du schéma et des contraintes	Schéma explicite Spark Bloc 2 ; Pydantic v2 Bloc 4
Fraîcheur	Donnée à jour par rapport au besoin	GSI timestamp Bloc 3 ; mode append streaming Bloc 2

8.4.2 Score de qualité Silver — Jobtech

Le `data_quality_score` (0–100) attribué à chaque enregistrement Silver Jobtech constitue une primitive de gouvernance quantitative. Il est calculé en pénalisant les valeurs manquantes sur les champs critiques (salaire, pays, niveau). Un score faible signale un enregistrement à surveiller avant exposition Gold :

```
score = 100
si salary_min IS NULL et salary_max IS NULL : score -= 30
si country_code IS NULL : score -= 20
si job_level_standardized = 'Not specified' : score -= 10
si contract_type IS NULL : score -= 10
score final : [0, 100]
```

Cette règle de scoring est la première étape d'un plan de remédiation : les enregistrements avec `score < 50` sont logués et exclus de la couche Gold par défaut. L'étape suivante serait un tableau de bord de suivi qualité par source, permettant d'identifier les sources dégradées et de déclencher un nettoyage ciblé.

8.4.3 Plan de remédiation

Face à une dégradation de qualité identifiée, le plan de remédiation suit quatre étapes :

1. **Détection** : alerte automatique si le score moyen d'une source descend sous un seuil défini (ex. score moyen < 60 pour Adzuna).
2. **Isolation** : les enregistrements dégradés sont conservés en Bronze mais exclus de la Silver en production.
3. **Investigation** : analyse de la cause racine — changement de schéma source, panne API, données corrompues.
4. **Rejouer** : une fois la source corrigée, rejouer uniquement l'étape concernée (idempotence garantie par la déduplication).

8.5 Conformité et réglementation

8.5.1 RGPD et données personnelles

Le Règlement Général sur la Protection des Données (RGPD) s'applique dès qu'une donnée permet d'identifier une personne physique, directement ou indirectement. Dans nos projets :

Projet	Données potentiellement personnelles	Traitement
project_cloud (départs)	Position GPS des voyageurs	Agrégation seule, pas de suivi individuel
Jobtech	Données d'entreprises (non-personnelles)	Pas de donnée individuelle
data_science_project	Capteurs machine (non-personnelles)	Pas de donnée individuelle

Pour project_cloud, les flux de départs ferroviaires contiennent des informations sur les mouvements des voyageurs. Bien que le projet agrège ces données (pas de suivi individuel), une analyse d'impact sur la protection des données (AIPD) serait requise en production pour qualifier le risque RGPD. Les mesures techniques (chiffrement KMS, rôles IAM restrictifs, rétention limitée) constituent la base de la réponse technique à cette analyse.

8.5.2 Traçabilité et auditabilité

L'auditabilité est garantie par trois mécanismes transversaux dans nos projets :

- **Horodatage systématique** : chaque enregistrement Bronze et Silver porte un `ingestion_timestamp` permettant de reconstituer l'état du système à tout instant.
- **Idempotence des pipelines** : la déduplication sur clé primaire garantit que rejouer un pipeline ne crée pas d'enregistrements fantômes, ce qui simplifie les audits de cohérence.
- **Vernonnage des modèles ML** : les artefacts `best_model_{task}.joblib` sont versionnés par tâche, permettant de retracer quelle version du modèle a produit quelle prédiction.

8.6 Stratégie de sécurité des données

8.6.1 Matrice de sécurité par couche

La stratégie de sécurité s'articule autour de la matrice CIA (Confidentialité, Intégrité, Disponibilité) appliquée à chaque couche de l'architecture :

Couche	Confidentialité	Intégrité	Disponibilité
Bronze (S3)	Chiffrement SSE-S3	Versioning S3	Réplication multi-zone
Silver (MySQL)	SSL connexions	Contraintes UNIQUE, FK	Pool connexions + retry
Silver (DynamoDB)	IAM restrictif	Transactions DynamoDB	Mode on-demand
Gold (API)	Rôle IAM read-only	Données pré-calculées	Lambda serverless
Modèles ML	Artefacts joblib versionnés	Hash de contrôle	Stockage local + backup

8.6.2 Politique d'accès aux données

La politique d'accès suit le principe du *need-to-know* : chaque acteur n'accède qu'aux données nécessaires à son rôle. En pratique, cela se traduit par :

- Les Data Analysts accèdent uniquement à la couche Gold via l'API (lecture seule).
- Les Data Engineers accèdent à Bronze et Silver pour les opérations de pipeline, mais pas directement à la production Gold.

- Les Data Scientists accèdent au dataset d'entraînement et aux artefacts ML, mais pas aux données nominatives s'il en existe.
- Les Ops ont accès aux logs et métriques CloudWatch, pas aux données métier.

Cette séparation est techniquement appliquée via les rôles IAM distincts (Lambda-api-role, ingestion-role, aggregation-role) décrits en Bloc 3, ce qui fait de la sécurité une propriété architecturale plutôt qu'une contrainte organisationnelle.

8.7 Synthèse des compétences démontrées

La gouvernance des données est une dimension transversale qui traverse l'ensemble de nos réalisations. Les éléments de Bloc 5 présents dans nos projets couvrent plusieurs sous-compétences attendues.

Qualité. Le score de qualité Jobtech, la déduplication systématique, l'imputation médiane et la stratification ML constituent des primitives de gouvernance de la qualité, directement exploitables pour un audit.

Sécurité. Les rôles IAM par fonctionnalité, le chiffrement en transit (TLS) et au repos (SSE-S3), et l'utilisateur non-root Docker démontrent une posture de sécurité cohérente avec les bonnes pratiques industrielles.

Traçabilité. L'horodatage systématique (microsecondes DynamoDB, partitionnement Year/Month/Day Spark, timestamp_ISO S3), l'idempotence des pipelines et le versionnage des modèles ML constituent une chaîne de traçabilité complète de la donnée brute à la prédiction.

Lien RNCP explicite. La compétence Bloc 5 exige de concevoir une stratégie de management et de gouvernance de données. Nos réalisations démontrent la maîtrise des dimensions clés : modèle RACI, catalogue de données, dimensions de qualité DMBOK, conformité RGPD, matrice CIA et politique d'accès. Ces éléments, transversaux à Bloc 1–Bloc 4, montrent que la gouvernance n'est pas traitée comme une couche ajoutée a posteriori, mais comme une contrainte intégrée dès la conception.

9 Conclusion

9.1 Fil conducteur — du stockage à la valeur métier

Ce rapport a présenté quatre projets couvrant l'intégralité du cycle de vie de la donnée dans un environnement Big Data, conformément au référentiel RNCP36739. Le fil conducteur qui les relie est la progression systématique de la donnée brute vers la valeur : chaque couche de l'architecture — Bronze, Silver, Gold — apporte un niveau de transformation supplémentaire, chaque service — Spark, DynamoDB, Lambda, scikit-learn — répond à une contrainte précise du problème.

Cette cohérence architecturale n'est pas accidentelle. Elle reflète une démarche de conception : avant tout choix technologique, la question du besoin est posée (quel pattern d'accès ? quel volume ? quelle latence ?) et la solution retenue est justifiée par rapport à ses alternatives. C'est cette capacité à argumenter les choix — plutôt qu'à simplement les énumérer — qui constitue le cœur de la compétence Data Engineering au niveau Master.

9.2 Synthèse des compétences acquises

Bloc	Compétence démontrée	Preuve concrète
Bloc 1	Architecture de stockage hybride (relationnelle + document + fichier columnar)	Pool MySQL pool_size=10, GSI DynamoDB O(1), Parquet/Snappy
Bloc 2	Pipeline distribué Spark + industrialisation CI/CD	Streaming structuré avec checkpoint, AQE, matrix GitHub Actions, Docker non-root
Bloc 3	Solutions cloud serverless et optimisation des coûts	Lambda + API Gateway, GSI DynamoDB, IAM moindre privilège, EventBridge
Bloc 4	Cycle ML complet de la préparation à la mise en service	ColumnTransformer, benchmark 4 modèles, F1=0.9775, FastAPI, Streamlit
Bloc 5	Gouvernance, qualité et sécurité transversales	Score qualité, RACI, RGPD, matrice CIA, IAM par rôle

9.3 Progression observée sur deux années

La progression sur deux années de formation se mesure à travers plusieurs axes.

Complexité des architectures. Le passage d'un pipeline ETL séquentiel (lecture, transformation, écriture) à une architecture distribuée multi-couches (Spark cluster, streaming avec checkpoint, Medallion Bronze/Silver/Gold) reflète une montée en compétence significative sur la conception à l'échelle.

Maîtrise des services cloud. La progression va de l'utilisation basique de boto3 (appels S3 simples) vers la conception d'une architecture serverless complète (Lambda, API Gateway, IAM, EventBridge, CloudWatch) avec une réflexion explicite sur les coûts et la sécurité.

Rigueur ML. L'approche évolue de l'entraînement d'un modèle unique vers un benchmark comparatif structuré avec métriques multiples (F1, ROC-AUC, PR-AUC), permutation importance, stratification et mise en service via API. La notion de data leakage (StandardScaler ajusté uniquement sur le train) est intégrée.

Industrialisation. L'adoption d'une chaîne CI/CD complète (lint bloquant, tests unitaires, coverage, matrix multi-version, containerisation Docker production-grade) traduit une maturité DevOps compatible avec les standards industriels.

9.4 Prise de recul — limites et axes d'amélioration

L'honnêteté intellectuelle impose de reconnaître les limites de nos réalisations.

Observabilité. Aucun des quatre projets ne dispose d'un tableau de bord de monitoring en temps réel (latence pipeline, taux d'erreur, volume ingéré). En production, l'absence d'observabilité est un risque opérationnel : un pipeline silencieusement défaillant n'est détecté que lors de la consommation des données aval. CloudWatch Metrics (Bloc 3) et Spark UI (Bloc 2) offrent des primitives, mais leur configuration en alerting automatique reste à finaliser.

Tests de charge. Les performances des pipelines sont mesurées sur les données disponibles (3 ans de vols US, dataset maintenance industrielle) mais pas sur des volumétries 10× ou 100× supérieures. Un test de charge (ex. Spark sur 100 Go de vols au lieu de 3 Go) permettrait de valider les hypothèses de scalabilité.

Dérive des données (data drift). Le modèle ML de maintenance prédictive est entraîné sur un dataset figé. En production réelle, les capteurs évoluent (remplacement, calibration), les conditions d'exploitation changent (nouvelles machines, nouveaux modes opératoires). Un mécanisme de détection de dérive (ex. comparaison des distributions des features entre entraînement et production via KL-divergence) est nécessaire pour déclencher un réentraînement au bon moment.

9.5 Ouverture

Ce rapport constitue le socle du dossier de certification RNCP36739. La soutenance orale permettra d'approfondir les choix architecturaux les plus significatifs — notamment la justification du GSI DynamoDB

face à l'alternative Elasticsearch, le choix d'AQE versus un partitionnement manuel, et le benchmark des modèles ML.

L'e-portfolio de compétences construit au fil des deux années — code source, artefacts de modèles, tableaux de bord, pipelines CI/CD — constitue la preuve concrète de l'acquisition des cinq blocs RNCP36739. La maîtrise technique démontrée dans ce rapport, combinée à la capacité d'argumentation et de prise de recul, positionne notre profil au niveau Expert en Ingénierie de Données attendu par le titre.

Références

- [1] France Compétences. *Fiche RNCP36739 – Expert en ingénierie de données*. Disponible sur : <https://www.francecompetences.fr/recherche/rncp/36739/> [consulté le 15 juin 2026].
- [2] Zaharia M., Xin R. S., Wendell P. et al. *Apache Spark : A Unified Engine for Big Data Processing*. *Communications of the ACM*, vol. 59, n°11, pp. 56–65, 2016. DOI : <https://doi.org/10.1145/2934664>
- [3] Apache Software Foundation. *Structured Streaming Programming Guide – Apache Spark*. Documentation officielle, version 3.x. Disponible sur : <https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html> [consulté le 15 juin 2026].
- [4] Apache Software Foundation. *Apache Parquet – Columnar Storage Format*. Disponible sur : <https://parquet.apache.org/> [consulté le 15 juin 2026].
- [5] Apache Software Foundation. *Apache Hive – Data Warehouse Software*. Disponible sur : <https://hive.apache.org/> [consulté le 15 juin 2026].
- [6] Oracle Corporation. *MySQL 8.0 Reference Manual*. Disponible sur : <https://dev.mysql.com/doc/refman/8.0/en/> [consulté le 15 juin 2026].
- [7] Amazon Web Services. *Amazon S3 – Object Storage Built to Retrieve Any Amount of Data*. Documentation officielle AWS. Disponible sur : <https://docs.aws.amazon.com/s3/> [consulté le 15 juin 2026].
- [8] DeCandia G., Hastorun D., Jampani M. et al. *Dynamo : Amazon’s Highly Available Key-value Store*. *ACM SIGOPS Operating Systems Review*, vol. 41, n°6, pp. 205–220, 2007. DOI : <https://doi.org/10.1145/1294261.1294281>
- [9] Amazon Web Services. *AWS Lambda – Serverless Compute Service*. Documentation officielle AWS. Disponible sur : <https://docs.aws.amazon.com/lambda/> [consulté le 15 juin 2026].
- [10] Amazon Web Services. *Amazon API Gateway – Build, Deploy and Manage APIs*. Documentation officielle AWS. Disponible sur : <https://docs.aws.amazon.com/apigateway/> [consulté le 15 juin 2026].
- [11] Amazon Web Services. *Boto3 – AWS SDK for Python*. Disponible sur : <https://boto3.amazonaws.com/v1/documentation/api/latest/index.html> [consulté le 15 juin 2026].
- [12] Open-Meteo. *Open-Meteo – Free Weather API*. Disponible sur : <https://open-meteo.com/> [consulté le 15 juin 2026].
- [13] Pedregosa F., Varoquaux G., Gramfort A. et al. *Scikit-learn : Machine Learning in Python*. *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. Disponible sur : <https://jmlr.org/papers/v12/pedregosa11a.html>
- [14] Ramírez S. *FastAPI – Modern, Fast Web Framework for Building APIs with Python*. Disponible sur : <https://fastapi.tiangolo.com/> [consulté le 15 juin 2026].
- [15] Docker Inc. *Docker – Accelerated Container Application Development*. Disponible sur : <https://docs.docker.com/> [consulté le 15 juin 2026].
- [16] GitHub Inc. *GitHub Actions – Automate Your Workflow from Idea to Production*. Disponible sur : <https://docs.github.com/en/actions> [consulté le 15 juin 2026].
- [17] Streamlit Inc. *Streamlit – A Faster Way to Build and Share Data Apps*. Disponible sur : <https://docs.streamlit.io/> [consulté le 15 juin 2026].
- [18] Christie T., Biewald L. et al. *Django REST Framework – Web APIs for Django*. Disponible sur : <https://www.django-rest-framework.org/> [consulté le 15 juin 2026].
- [19] Python Software Foundation. *Python 3 Reference Documentation*. Disponible sur : <https://docs.python.org/3/> [consulté le 15 juin 2026].